



MoonChunk

Podrecznik uzytkownika

Dokumentacja z perspektywy osoby tworzacej strony i treści

Ten dokument opisuje MoonChunk jako jezyk do budowania statycznych stron HTML. Nie skupia sie na wewnetrznej implementacji parsera ani na gramatyce ANTLR. Zamiast tego prowadzi czytelnika od podstawowych pojec programistycznych, przez skladnie jezyka, az do kompletnego projektu tworzonego krok po kroku.

Wersja: 1.0

Repozytorium: <https://github.com/dhunanyan/moonchunk>

Jezyk dokumentacji: polski

Jak czytac ten podrecznik

Ten podrecznik zostal napisany tak, aby mogla z niego skorzystac takze osoba, ktora nie miala wczesniej kontaktu z jezykami programowania. Z tego powodu kazdy nowy element jest przedstawiany w dwoch warstwach:

- najpierw pojawia sie intuicja: po co dana konstrukcja istnieje i jaki problem rozwiazuje,
- dopiero potem pokazana jest konkretna skladnia MoonChunk wraz z przykladami.

W wielu miejscach zobaczysz male fragmenty kodu. Czytaj je spokojnie, linia po linii. MoonChunk jest jezykiem celowo zwartym, wiec stosunkowo niewielka liczba konstrukcji pozwala zbudowac zaskakujaco duzo.

Spis treści

1	Czym jest MoonChunk	1
1.1	Idea języka	1
1.2	Co to znaczy, że MoonChunk jest językiem deklaratywno-imperatywnym	1
1.3	Dla kogo jest MoonChunk	2
1.4	Jak myśleć o projekcie w MoonChunk	3
1.5	MoonChunk a HTML	3
1.6	MoonChunk a klasyczne programowanie	4
1.7	Jakie problemy MoonChunk rozwiązuje najlepiej	4
1.8	Co znajdziesz dalej	4
2	Jak zacząć pracę z MoonChunk	5
2.1	Dwa poziomy pracy: użytkownik języka i autor języka	5
2.2	Najprostszy model użycia	5
2.3	Instalacja w projekcie użytkownika	6
2.4	Minimalny skrypt uruchamiający	6
2.5	Co robi <code>executeMoonChunkFile(...)</code>	7
2.6	Dlaczego warto mieć prosty runner	7
2.7	Pierwsze uruchomienie z przygotowanym plikiem	8
2.8	Najczęstsze pytania początkujących	8
2.9	Nastawienie, które pomaga	9
3	Pierwszy program	10

3.1	Po co zaczynac od najmniejszego mozliwego przykladu	10
3.2	Czytanie programu od gory do dolu	11
3.3	Dlaczego <code>moon(...)</code> jest takie wazne	12
3.4	Wersja rozbudowana o zmienne	12
3.5	Co dzieje sie podczas wykonania	13
3.6	Najwazniejsza roznica miedzy kodem a wynikiem	13
3.7	Typowe pomyly w pierwszym programie	14
3.8	Co warto zapamietac po tym rozdziale	14
4	Struktura programu	16
4.1	Dlaczego struktura jest wazna	16
4.2	Co moze znajdowac sie w pliku <code>.mncnk</code>	16
4.3	Czym jest <code>chunk</code>	17
4.4	Jedna nazwa, jedna odpowiedzialnosc	18
4.5	Wywołanie <code>moon(...)</code> jako punkt startowy	18
4.6	Import a wykonanie	19
4.7	Eksportowanie chunkow	19
4.8	Kolejnosc w pliku	20
4.9	Plik glowny i pliki pomocnicze	20
4.10	Myslenie warstwowe	21
4.11	Jak rozpoznać, ze struktura robi sie zla	21
5	Strony i HTML	22
5.1	Strona jako finalny produkt	22
5.2	Jak rozumiec trase strony	22
5.3	Po co to ulatwienie	23
5.4	Blok <code>content</code>	23
5.5	Dynamiczne wstawianie wartosci	24
5.6	Atrybuty HTML	24

5.7	Czy w <code>content</code> można mieszać HTML i logikę	24
5.8	Petle i warunki wewnątrz strony	25
5.9	Wiecej niż jeden blok <code>content</code>	26
5.10	HTML jako wynik, nie jako cel sam w sobie	27
5.11	Praktyczna rada dotycząca czytelności	27
6	Metadata strony	28
6.1	Po co w ogóle są metadata	28
6.2	Podstawowa składnia metadata	28
6.3	Dlaczego to rozwiązanie jest wygodne	29
6.4	Metadata wspólne i lokalne	29
6.5	Kategorie wspieranych metadata	30
6.6	Co się dzieje, gdy pole jest puste	32
6.7	Metadata a treść strony	33
6.8	Przykład rozsądnego zestawu startowego	33
6.9	Kiedy wydzielić metadata do osobnego chunku	34
6.10	O czym warto pamiętać na koniec	34
7	Zmienne, stałe i typy	35
7.1	Po co istnieją zmienne	35
7.2	<code>let</code> i <code>const</code>	35
7.3	Czy <code>let</code> musi mieć wartość od razu	36
7.4	Czy <code>const</code> może być puste na starcie	37
7.5	Typy w MoonChunk	37
7.6	Typy liczbowe	38
7.7	Typ <code>bool</code>	38
7.8	Typ <code>string</code>	39
7.9	Typy złożone: <code>array</code> , <code>object</code> , <code>dict</code>	39
7.10	Typy specjalne: <code>null</code> i <code>undefined</code>	40

7.11	Typy <code>unknown</code> i <code>any</code>	40
7.12	Typ <code>void</code>	41
7.13	Deklaracja typu jawnie i niejawnie	41
7.14	Zgodność typu przy przypisaniu	41
7.15	Globalne zmienne w <code>env</code>	42
7.16	Praktyczne wskazówki dotyczące typów	42
7.17	Najczęstsze błędy początkujących	42
8	Wyrażenia i operatory	44
8.1	Czym jest wyrażenie	44
8.2	Operatory arytmetyczne	44
8.3	Kolejność wykonywania działań	45
8.4	Dzielenie przez zero i modulo przez zero	45
8.5	Operatory porównania	46
8.6	Operatory logiczne	46
8.7	Operator warunkowy	47
8.8	Lączenie tekstu	47
8.9	Przypisanie jako wyrażenie i zwykle przypisanie	47
8.10	Operator inkrementacji	48
8.11	Wyrażenia w HTML	48
8.12	Wyrażenia w indeksowaniu i dostępie do pól	48
8.13	Kiedy wyrażenie jest zbyt złożone	49
9	Rzutowanie typów	50
9.1	Po co w ogóle rzutować typy	50
9.2	Rzutowanie w stylu <code>(typ)wartosc</code>	50
9.3	Rzutowanie przez <code>as</code>	51
9.4	Rzutowanie liczb i tekstu	51
9.5	Rzutowanie do <code>bool</code>	51

9.6	Jak zachowuje sie liczba rzutowana do <code>bool</code>	52
9.7	Przechodzenie przez <code>unknown</code> i <code>any</code>	52
9.8	Rzutowanie obiektow, slownikow i tablic	53
9.9	Rzutowanie do <code>null</code> i <code>undefined</code>	53
9.10	Co nie jest rzutowaniem	53
9.11	Kiedy rzutowac, a kiedy lepiej nie	54
9.12	Rzutowanie w praktyce strony	54
10	Instrukcje warunkowe	55
10.1	Po co programowi warunki	55
10.2	Najprostsza forma <code>if</code>	55
10.3	Warunek musi byc typu <code>bool</code>	56
10.4	<code>if</code> z <code>else</code>	56
10.5	Warunki a HTML	56
10.6	Warunki zlozone	57
10.7	Zagniezdzone warunki	58
10.8	If a zakresy nazw	58
10.9	Kiedy uzywac <code>if</code> , a kiedy operatora <code>?:</code>	58
10.10	Typowe zastosowania w stronach	59
10.11	Najczestsze bledy przy warunkach	59
11	Petle: powtarzanie pracy bez kopiowania kodu	61
11.1	Po co istnieja petle	61
11.2	Petla <code>for</code>	61
11.3	Budowa petli <code>for</code> krok po kroku	62
11.4	Czy aktualizacja w <code>for</code> musi byc <code>i++</code>	62
11.5	Petla <code>while</code>	63
11.6	Warunek petli musi byc typu <code>bool</code>	63
11.7	Petle a generowanie HTML	63

11.8 <code>break</code> i <code>continue</code>	64
11.9 Gdzie wolno używać <code>break</code> i <code>continue</code>	65
11.10 Petle zagnieżdżone	65
11.11 Petle a zakres nazw	65
11.12 Najczęstsze błędy przy petlach	66
11.13 Kiedy wybrać <code>for</code> , a kiedy <code>while</code>	66
12 Zasięgi, bloki i widoczność nazw	67
12.1 Dlaczego zasięg jest tak ważny	67
12.2 Najprostsza intuicja	67
12.3 Redeklaracja a zwykłe przypisanie	68
12.4 Granice, przy których ta sama nazwa jest dozwolona	68
12.5 Osobne chunki jako osobne granice	68
12.6 Funkcja jako granica zasięgu	69
12.7 Jawny blok <code>{ ... }</code>	69
12.8 Dlaczego to rozwiązanie jest przydatne	70
12.9 Ciała <code>if</code> , <code>for</code> i <code>while</code> są bardziej rygorystyczne	70
12.10 Odczyt wartości z nadrzędnego zakresu przez <code>parent::</code>	71
12.11 Kiedy nie używać <code>parent::</code>	71
12.12 Globalne wartości a zasięgi lokalne	72
12.13 Najczęstsze błędy związane z zasięgami	72
12.14 Złota zasada czytelności	72
13 Funkcje i rekurencja	73
13.1 Po co istnieją funkcje	73
13.2 Klasyczna deklaracja funkcji	73
13.3 Parametry funkcji	74
13.4 Instrukcja <code>return</code>	74
13.5 Funkcje bez zwracanej wartości	75

13.6	Funkcje a HTML	75
13.7	Funkcje jako granica zasięgu	75
13.8	Rekurencja	76
13.9	Warunek stopu w rekurencji	76
13.10	Rekurencja do przechodzenia po kolejnych krokach	76
13.11	Zapis strzałkowy	77
13.12	Funkcje jako wartości	77
13.13	Najczęstsze błędy przy funkcjach	78
13.14	Kiedy wydzielić funkcje	78
14	Importy, @include i składanie projektu z wielu plików	79
14.1	Dlaczego w ogóle dzielić projekt na pliki	79
14.2	Importy muszą stać na samej górze pliku	79
14.3	Eksportowany chunk	80
14.4	Import nazwany	80
14.5	Import przestrzeni nazw	80
14.6	Samo import niczego jeszcze nie uruchamia	81
14.7	Do czego służy @include	81
14.8	@include powinno pojawiać się na początku chunku	81
14.9	Import nazwany a import przez namespace	81
14.10	Lancuch importów	82
14.11	Cykliczne zależności	82
14.12	Punkt wejścia nadal musi być jawny	82
14.13	Typowy wzorec modularnego projektu	83
14.14	Najczęstsze błędy przy importach	83
14.15	Jak myśleć o @include	84
15	Tablice, obiekty, dane z JSON i model rekordów	85
15.1	Dlaczego dane złożone są potrzebne	85

15.2	Tablice	85
15.3	Czy tablica musi być jednorodna	86
15.4	Obiekty i słowniki	86
15.5	Dostęp do pól przez kropkę	86
15.6	Dostęp przez nawiasy kwadratowe	87
15.7	Klucze numeryczne i nietypowe	87
15.8	Modyfikowanie pól i elementów	87
15.9	Tablice i obiekty jako model rekordów	88
15.10	Wczytywanie danych z pliku JSON przez <code>data(...)</code>	88
15.11	Przykładowy plik JSON	88
15.12	Przykład pracy z danymi JSON	89
15.13	Kiedy używać <code>object</code> , a kiedy <code>dict</code>	89
15.14	Typowe błędy przy danych złożonych	90
15.15	Dobra praktyka: dane najpierw, HTML potem	90
16	Print, diagnostyka i czytanie błędów	91
16.1	Po co istnieje <code>print(...)</code>	91
16.2	Najprostsze użycie	91
16.3	Wypisywanie pustej linii	91
16.4	Co można wypisać	92
16.5	Print jako narzędzie edukacyjne	92
16.6	Jak wygląda dobry log	93
16.7	Błędy składniowe a błędy wykonania	93
16.8	Jak czytać komunikat diagnostyczny	94
16.9	Typowe komunikaty i ich znaczenie	94
16.10	Strategia debugowania krok po kroku	95
16.11	Jak odróżnić błąd w logice od błędu w danych	95
16.12	Dobra praktyka: loguj stan, nie emocje	95
16.13	Kiedy usuwać logi	96

17 Dobre praktyki pracy z MoonChunk	97
17.1 Po co w ogóle mówić o praktykach	97
17.2 Oddzielaj dane od prezentacji	97
17.3 Nazywaj rzeczy tak, jak je rozumiesz	98
17.4 Trzymaj metadata razem	98
17.5 Używaj funkcji do logiki, nie do wszystkiego	98
17.6 Ograniczaj magię w wyrażeniach	99
17.7 Utrzymuj pliki małe i sensowne	99
17.8 Używaj <code>print(...)</code> świadomie	99
17.9 Nie nadużywaj globali	99
17.10 Wybieraj prostszy typ, jeśli wystarczy	100
17.11 Zanim rozbudujesz, zbuduj mały pionowy przekroj	100
17.12 Komentuj tam, gdzie jest decyzja, nie tam, gdzie jest oczywistość	100
17.13 Buduj projekt tak, jakby miał urosnąć	101
18 Projekt od zera: budujemy mały serwis krok po kroku	102
18.1 Cel projektu	102
18.2 Struktura plików projektu	102
18.3 Dane wpisów	103
18.4 Wspólne metadata	104
18.5 Wspólna sekcja powitalna	104
18.6 Logika bloga	105
18.7 Plik główny	106
18.8 Co zrobi ten projekt	106
18.9 Minimalny runner do tego projektu	107
18.10 Jak rozwijać ten projekt dalej	107
18.11 Czego ten projekt uczy	108
18.12 Ostatnia rada praktyczna	108

19 Wzorce uzycia i gotowe przepisy	109
19.1 Po co w ogole wzorce	109
19.2 Wzorzec: strona glowna z sekcjami	109
19.3 Wzorzec: lista kart	110
19.4 Wzorzec: stan pusty	110
19.5 Wzorzec: filtr publikacji	111
19.6 Wzorzec: funkcja do etykiety	111
19.7 Wzorzec: wspolne metadata plus lokalne nadpisanie	112
19.8 Wzorzec: oblicz najpierw, pokaz potem	112
19.9 Wzorzec: rekurencyjny spacer po etapach	113
20 Antywzorce i pulapki	114
20.1 Po co poznawac antywzorce	114
20.2 Antywzorzec: wszystko w jednym pliku	114
20.3 Antywzorzec: kopiowanie metadata	114
20.4 Antywzorzec: zbyt duzo logiki w <code>content</code>	115
20.5 Antywzorzec: naduzywanie <code>unknown</code> i <code>any</code>	115
20.6 Antywzorzec: brak warunku stopu w rekurencji	115
20.7 Antywzorzec: nieczytelne nazwy	115
20.8 Antywzorzec: traktowanie <code>if</code> jako miejsca do swobodnego cieniowania	115
20.9 Antywzorzec: import bez realnego uzycia	116
20.10 Antywzorzec: brak strategii debugowania	116
20.11 Antywzorzec: projekt budowany od konca	116
21 FAQ: najczestsze pytania uzytkownikow	117
21.1 Czy jeden plik <code>.mncnk</code> moze generowac wiele stron	117
21.2 Czy import od razu generuje HTML	117
21.3 Czy musze podawac <code>.html</code> w trasie strony	117
21.4 Czy moze pominac wiodacy ukośnik w trasie	117

21.5 Czy mogę użyć <code>content</code> poza <code>page</code>	118
21.6 Czy <code>let</code> bez wartości jest bezpieczne	118
21.7 Czy mogę zmieniać <code>const</code>	118
21.8 Czy mogę mieć te same nazwy w wielu miejscach	118
21.9 Czy MoonChunk ma osobny typ rekordu albo krotki	118
21.10 Czy dane z JSON są tylko do odczytu	118
21.11 Czy mogę wypisać funkcję albo chunk przez <code>print</code>	119
21.12 Czy pętle muszą mieć licznik typu <code>int</code>	119
21.13 Czy <code>break</code> i <code>continue</code> działają wszędzie	119
21.14 Czy muszę wszędzie podawać typy	119
21.15 Czy muszę znać parser i ANTLR, żeby korzystać z MoonChunk	119
22 Ćwiczenia i propozycje samodzielnej pracy	120
22.1 Dlaczego warto robić ćwiczenia	120
22.2 Ćwiczenie 1: prosta strona wizytówka	120
22.3 Ćwiczenie 2: lista kart	120
22.4 Ćwiczenie 3: filtr opublikowanych wpisów	121
22.5 Ćwiczenie 4: funkcja etykietująca	121
22.6 Ćwiczenie 5: wczytanie JSON	121
22.7 Ćwiczenie 6: wspólne metadane	121
22.8 Ćwiczenie 7: rekurencja	121
22.9 Ćwiczenie 8: jawny blok zasięgu	122
22.10 Jak pracować z ćwiczeniami	122
23 Jak rozwijać projekt od małego do średniego serwisu	123
23.1 Etap 1: jedna strona i jeden chunk	123
23.2 Etap 2: wspólne metadane	123
23.3 Etap 3: funkcje pomocnicze	123
23.4 Etap 4: dane z JSON	123

23.5	Etap 5: podział na pliki	124
23.6	Etap 6: wzorce projektowe	124
23.7	Etap 7: czyszczenie i stabilizacja	124
23.8	Etap 8: świadome skalowanie	125
23.9	Zasada, która ratuje projekt	125
A	Szybka ściaga składni	126
A.1	Podstawowy szkielet	126
A.2	Zmienne i stałe	126
A.3	Warunki	127
A.4	Petle	127
A.5	Funkcje	127
A.6	Importy	127
A.7	Include	127
A.8	Tablice i obiekty	128
A.9	Rzutowanie	128
A.10	Globale	128
B	Referencja metadata	129
B.1	Praktyczna rada	131
C	Słownik pojęć	132

Rozdział 1

Czym jest MoonChunk

1.1 Idea języka

MoonChunk jest językiem do generowania statycznych stron HTML. Oznacza to, że piszemy kod w plikach z rozszerzeniem `.mncnk`, a wynikiem działania programu są zwykle gotowe pliki `.html`. Taki model jest bardzo wygodny wtedy, gdy chcemy:

- zbudować małą lub średnią stronę internetową,
- tworzyć wiele podobnych podstron bez kopiowania kodu,
- rozdzielić treść, logikę i metadane,
- wykorzystywać pętle, warunki i funkcje do składania strony,
- pracować na danych zapisanych np. w plikach JSON.

MoonChunk nie jest klasycznym językiem ogólnego przeznaczenia takim jak C, JavaScript czy Python. Jego główny cel jest bardziej wyspecjalizowany: **opisanie, jak mają powstać strony HTML**. W praktyce daje to dwie korzyści. Po pierwsze, kod jest krótszy i bardziej skupiony na strukturze strony. Po drugie, łatwiej jest utrzymać porządek, gdy witryna zaczyna rosnąć.

1.2 Co to znaczy, że MoonChunk jest językiem deklaratywno-imperatywnym

W MoonChunk spotykają się dwa sposoby myślenia o programie.

Czesc deklaratywna

Deklarujemy, *co* ma sie pojawic na stronie:

- jaka jest trasa strony,
- jakie ma miec metadata,
- jaki HTML ma byc wygenerowany.

Czesc imperatywna

Opisujemy tez, *jak* dojsc do wyniku:

- obliczamy wartosci,
- wykonujemy warunki `if`,
- przechodzimy po elementach w petli,
- wywoujemy funkcje,
- importujemy fragmenty z innych plikow.

To polaczenie jest szczegolnie wygodne przy stronach, bo pozwala zarowno opisac strukture dokumentu, jak i kontrolowac logike potrzebna do jego wygenerowania.

1.3 Dla kogo jest MoonChunk

MoonChunk jest dobrym wyborem dla kilku grup odbiorcow.

Dla osob uczacych sie jezykow i interpreterow

Jezyk ma niewielka powierzchnie skladniowa, a jednoczesnie zawiera klasyczne pojecia programistyczne: zmienne, typy, funkcje, rekurencje, petle, importy oraz zakresy widocznosci. To sprawia, ze dobrze nadaje sie do nauki.

Dla osob tworzacych statyczne strony

Jesli chcesz wygenerowac zestaw stron produktowych, portfolio, dokumentacje, mini-bloga albo zestaw landing pages, MoonChunk daje Ci duzo porzadku przy malej ilosci kodu.

Dla osob, ktore nie chca duzego frameworka

Nie kazdy projekt potrzebuje Reacta, Vite, rozbudowanego CMS-a albo serwera po stronie backendu. Czasem wystarczy prosty proces: wejscie w postaci danych i plikow `.mncnk`, wyjscie w postaci gotowego HTML.

1.4 Jak myslec o projekcie w MoonChunk

Najprostszy model mentalny wyglada tak:

1. tworzysz plik `.mncnk`,
2. definiujesz w nim jeden lub kilka `chunk`-ow,
3. w wybranym miejscu uruchamiasz punkt wejscia przez `moon(...)`,
4. w trakcie wykonania MoonChunk tworzy strony `page`,
5. kazda strona sklada HTML z blokow `content`,
6. gotowe dokumenty trafiaja do katalogu wyjsciowego.

To podejscie warto zapamietac. W MoonChunk nie piszesz programu po to, aby stale reagowal na klikniecia uzytkownika w przegladarce. Piszysz go po to, aby **wygenerowac finalne pliki**. Oznacza to, ze duza czesc logiki wykonuje sie w momencie budowania, a nie w chwili ogladania strony przez odwiedzajacego.

1.5 MoonChunk a HTML

MoonChunk nie zastepuje HTML. On go **wytwarza**. To duza roznicza. Kiedy tworzysz blok `content`, w praktyce opisujesz fragment przyszlego HTML-a. Mozesz wpisac tam zwykle znaczniki, a w miejscach dynamicznych umiescic wyrazenia w nawiasach klamrowych.

```
1  content {  
2    <h1>{title}</h1>  
3    <p>{description}</p>  
4  };
```

Listing 1.1: Fragment dynamicznego HTML w MoonChunk

Po wykonaniu programu `title` i `description` zostana zamienione na konkretne wartosci, a wynik trafi do pliku HTML.

1.6 MoonChunk a klasyczne programowanie

Jesli nie programowales wczesniej, kilka pojec moze brzmiec obco. Oto najwazniejsze z nich w uproszczonej formie:

- **zmienna** to nazwana wartosc, np. tytul strony albo liczba odwiedzin,
- **typ** opisuje, jakiego rodzaju jest wartosc, np. liczba, tekst albo wartosc logiczna,
- **warunek** pozwala wybrac jedna z dwoch drog wykonania,
- **petla** pozwala powtorzyc podobne dzialanie wiele razy,
- **funkcja** to fragment logiki, ktory mozna wywolywac wielokrotnie,
- **zakres** odpowiada za to, gdzie dana zmienna jest widoczna.

W kolejnych rozdzialach kazde z tych pojec zostanie omowione spokojnie i od podstaw.

1.7 Jakie problemy MoonChunk rozwiazuje najlepiej

MoonChunk szczegolnie dobrze sprawdza sie w nastepujacych scenariuszach:

- generowanie wielu podobnych stron na podstawie danych,
- utrzymywanie wspolnych metadata dla calego serwisu,
- skladanie strony z importowanych fragmentow,
- oddzielenie warstwy danych od warstwy HTML,
- uczenie sie budowy jezyka i interpretera na praktycznym przykladzie.

Jesli potrzebujesz ogromnej aplikacji klienckiej z rozbudowanym stanem w przegladarce, MoonChunk sam w sobie nie zastapi takiego ekosystemu. Natomiast jako narzedzie do budowania gotowych dokumentow HTML jest bardzo wygodny i czytelny.

1.8 Co znajdziesz dalej

W nastepnym rozdziale przejdziemy od idei do praktyki. Zobaczysz, jak rozpoczac prace z MoonChunk jako zwykly uzytkownik, bez zaglabiania sie w szczegoly parsera czy procesu budowania samego jezyka.

Rozdział 2

Jak zacząć pracę z MoonChunk

2.1 Dwa poziomy pracy: użytkownik języka i autor języka

To bardzo ważne rozróżnienie. W repozytorium MoonChunk można zobaczyć wiele poleceń związanych z budowaniem projektu, generowaniem parsera czy uruchamianiem testów. Są one potrzebne głównie wtedy, gdy rozwijasz sam *MoonChunk jako projekt*.

Ten podręcznik ma inny cel. Interesuje nas osoba, która chce **używać języka**, a niekoniecznie rozwijać jego implementację. Z punktu widzenia takiego użytkownika najważniejsze jest co innego:

- jak napisać plik `.mncnk`,
- jak uruchomić jego wykonanie,
- gdzie pojawia się wynik,
- jak odczytywać błędy,
- jak organizować większy projekt.

2.2 Najprostszy model użycia

Obecny model pracy z MoonChunk najłatwiej opisać tak:

1. instalujesz pakiet `moonchunk` w projekcie Node.js,
2. tworzysz swoje pliki `.mncnk`,

3. uruchamiasz prosty skrypt JavaScript albo TypeScript,
4. skrypt wywołuje funkcję `executeMoonChunkFile(...)`,
5. MoonChunk generuje pliki HTML.

To podejście ma jedną dużą zaletę: nie musisz znać wewnętrznej struktury repozytorium, a jednocześnie masz pełną kontrolę nad tym, kiedy i z jakiego pliku rozpoczynasz budowanie strony.

2.3 Instalacja w projekcie użytkownika

W zwykłym projekcie Node.js możesz zainstalować MoonChunk z rejestru npm. Przykładowo:

```
1 npm install moonchunk
```

Listing 2.1: Instalacja pakietu MoonChunk w projekcie użytkownika

Jeśli korzystasz z Yarn:

```
1 yarn add moonchunk
```

Listing 2.2: Instalacja pakietu przez Yarn

Warto od razu przygotować prosty katalog projektu, na przykład taki:

```
1 my-site/  
2   content/  
3     site.mcnk  
4     posts.json  
5   run.js  
6   package.json
```

Listing 2.3: Przykładowy układ plików projektu

2.4 Minimalny skrypt uruchamiający

Ponieważ najważniejszy interfejs dla użytkownika jest obecnie programistyczny, najprościej utworzyć mały plik `run.js`:

```
1 const { executeMoonChunkFile } = require("moonchunk");  
2  
3 const result = executeMoonChunkFile("./content/site.mcnk");  
4
```

```
5  if (!result.ok) {  
6      console.error("Wystąpił błąd:");  
7      console.error(result.diagnostics);  
8      process.exit(1);  
9  }  
10  
11  console.log("Wygenerowane pliki:");  
12  console.log(result.generatedFiles || []);
```

Listing 2.4: Najprostszy runner MoonChunk w JavaScript

Ten skrypt jest prosty, ale w zupełności wystarczy do codziennego użycia. Nie musisz wiedzieć nic o ANTLR, parserze ani o wewnętrznej architekturze projektu.

2.5 Co robi `executeMoonChunkFile(...)`

Funkcja `executeMoonChunkFile(...)` bierze ścieżkę do pliku `.mncnk`, czyta jego zawartość, uruchamia parser i interpreter, a następnie zwraca obiekt z wynikiem. Najważniejsze pola tego obiektu to:

- `ok` – informacja, czy wykonanie zakończyło się sukcesem,
- `generatedFiles` – lista wygenerowanych plików HTML,
- `diagnostics` – lista komunikatów diagnostycznych, jeśli wystąpiły błędy,
- `output` – dodatkowy log związany z wykonaniem.

Dla zwykłego użytkownika najczęściej wystarczy sprawdzenie pola `ok`, a potem odczytanie `generatedFiles` albo `diagnostics`.

2.6 Dlaczego warto mieć prosty runner

Mozna zapytać: po co mi dodatkowy plik `run.js`? Powód jest praktyczny. Dzięki niemu w jednym miejscu decydujesz:

- jaki plik `.mncnk` jest punktem wejścia,
- co zrobić po sukcesie,
- jak wyświetlać błędy,

- czy chcesz generować kilka projektów jeden po drugim.

To bardzo wygodny wzorzec, zwłaszcza gdy projekt rośnie.

2.7 Pierwsze uruchomienie z przygotowanym plikiem

Kiedy masz już skrypt uruchamiający oraz plik `site.mcnk`, wystarczy wywołać:

```
1 node run.js
```

Listing 2.5: Uruchomienie generatora

Jeśli wszystko jest poprawne, MoonChunk wygeneruje gotowe pliki HTML. Gdzie trafi wynik? To zależy od tego, jaką ścieżkę podasz w instrukcji `output`. O niej powiemy szerzej w kolejnych rozdziałach.

2.8 Najczęstsze pytania początkujących

Czy muszę znać JavaScript, żeby używać MoonChunk

Nie w pełnym zakresie. Wystarczy Ci bardzo prosty skrypt startowy, który można raz przygotować i potem prawie do niego nie wracać. Sama praca nad treścią i logiką stron odbywa się już w MoonChunk.

Czy MoonChunk działa tylko z Node.js

Obecny sposób użycia jest oparty o Node.js, bo właśnie tam wykonywany jest pakiet. Dla osoby korzystającej z języka oznacza to po prostu tyle, że potrzebuje środowiska, które uruchomi skrypt startowy.

Czy jedna strona to jeden plik

Niekoniecznie. Możesz mieć jeden plik z wieloma `chunk`-ami, wiele plików z importami, a nawet wiele punktów wejścia przez `moon(...)`. Oznacza to, że strukturę projektu możesz dopasować do swojej skali pracy.

2.9 Nastawienie, które pomaga

MoonChunk najłatwiej poznawac małymi krokami. Dobra kolejność wygląda tak:

1. najpierw zbuduj jedną, bardzo prostą stronę,
2. potem dodaj zmienne,
3. następnie dodaj warunek i pętle,
4. dopiero potem przejdź do funkcji, importów i danych z JSON.

W kolejnym rozdziale przejdziemy przez pierwszy kompletny program, żeby zobaczyć, jak te wszystkie elementy łączą się w jedną całość.

Rozdział 3

Pierwszy program

3.1 Po co zaczynac od najmniejszego mozliwego przykladu

Kiedy poznajemy nowy jezyk, bardzo latwo jest od razu zobaczyc duzy przyklad, przestraszyc sie liczby elementow i stracic intuicje. Dlatego zaczniemy od programu, ktory robi tylko jedna rzecz: generuje pojedyncza strone glowna.

Oto minimalny przyklad:

```
1  chunk "Main" {
2    output: "./dist";
3
4    page "" {
5      content {
6        <h1>Witaj w MoonChunk</h1>
7        <p>To jest moja pierwsza strona.</p>
8      };
9    };
10 };
11
12 moon(Main);
```

Listing 3.1: Minimalny program w MoonChunk

Jest tu zaledwie kilka elementow, ale juz teraz widzimy prawie caly podstawowy model pracy z jezykiem.

3.2 Czytanie programu od góry do dołu

Przejdźmy przez kod linia po linii.

```
chunk "Main"{ ... };
```

`chunk` jest podstawowym kontenerem logiki i treści w MoonChunk. Można o nim myśleć jak o module albo opakowaniu na fragment programu. Wewnątrz `chunk`-a umieszcza się instrukcje związane z generowaniem stron, definicje funkcji, zmienne, metadata oraz importowane fragmenty przez `@include`.

Nazwa `"Main"` jest nazwą logiczną. Nie jest to nazwa pliku HTML. To nazwa obiektu, do którego potem odwołujemy się przez `moon(Main)`.

```
output: "./dist";
```

Instrukcja `output` mówi, gdzie mają trafić wygenerowane pliki. W tym przykładzie wyniki zostaną zapisane w katalogu `dist`. Jeśli wygenerujemy stronę główną, prawdopodobnie zobaczymy tam plik `index.html`.

```
page ""{ ... };
```

`page` oznacza pojedynczą stronę, która ma zostać wygenerowana. Pusty napis `""` oznacza trasę główną. W praktyce przekłada się to na `index.html`.

To bardzo ważny wzorec:

- `page ""` to strona główna,
- `page "about"` to zwykle `about.html`,
- `page "blog/post-1"` to zwykle `blog/post-1.html`.

```
content { ... };
```

Blok `content` przechowuje HTML, który trafi do głównej części dokumentu. To miejsce, w którym najczęściej wpisuje się znaczniki takie jak `<h1>`, `<p>`, `<section>`, `<article>` czy `<ul>`.

```
moon(Main);
```

Ta linia uruchamia wykonanie wybranego `chunk`-a. Bez niej program może zostać poprawnie sparsowany, ale nic nie zostanie wygenerowane, ponieważ nie wskazaliśmy punktu wejścia.

3.3 Dlaczego `moon(...)` jest takie ważne

W wielu językach program zaczyna się automatycznie od jednej, uprzywilejowanej funkcji. MoonChunk działa trochę inaczej. Możesz mieć w jednym projekcie wiele `chunk`-ów, a `moon(...)` jawnie wskazuje, który z nich ma zostać wykonany.

Dzięki temu:

- łatwo pracować na różnych wariantach projektu,
- można generować kilka zestawów stron w jednym przebiegu,
- importowane fragmenty nie uruchamiają się same z siebie,
- kod staje się bardziej przewidywalny.

To niepozorna konstrukcja, ale bardzo porządkuje duże projekty.

3.4 Wersja rozbudowana o zmienne

Dodajmy teraz dwie proste zmienne i wykorzystajmy je w HTML:

```
1 chunk "Main" {
2   output: "./dist";
3
4   page "" {
5     let title = "Moja pierwsza strona";
6     let intro = "Ten tekst pochodzi ze zmiennej.";
7
8     content {
9       <h1>{title}</h1>
10      <p>{intro}</p>
11    };
12  };
13 };
14
```

```
15 moon(Main);
```

Listing 3.2: Pierwszy program z danymi dynamicznymi

W tym przykładzie `title` i `intro` nie są wpisane na sztywno w HTML-u. Najpierw otrzymują wartość, a potem są wstawiane do treści strony przez zapis `{title}` i `{intro}`.

3.5 Co dzieje się podczas wykonania

Gdy MoonChunk uruchamia taki program, można to opisać w następujących krokach:

1. odczytuje plik `.mncnk`,
2. rozpoznaje `chunk`-i, strony, zmienne i pozostałą logikę,
3. wykonuje `moon(Main)`,
4. wchodzi do wnętrza `chunk "Main"`,
5. dochodzi do strony `page ""`,
6. tworzy zmienne `title` i `intro`,
7. składa HTML z bloku `content`,
8. zapisuje wynik do pliku wyjściowego.

Jeśli rozumiesz te osiem punktów, rozumiesz już podstawowy cykl życia programu MoonChunk.

3.6 Najważniejsza różnica między kodem a wynikiem

Dobrze jest rozdzielić w głowie dwie rzeczy:

- **kod MoonChunk** – to instrukcje, które piszesz,
- **wygenerowany HTML** – to finalny wynik po wykonaniu instrukcji.

Na przykład taki kod:

```
1 let name = "Alicja";
2 content {
3   <p>Czesc, {name}!</p>
4 };
```

Listing 3.3: Kod z interpolacją wartości

może zamienić się na wynikowy HTML:

```
1 <p>Czesc, Alicja!</p>
```

Listing 3.4: Przykładowy wynik po wykonaniu

Ta umiejętność rozróżniania etapu *pisania programu* od etapu *wygenerowania wyniku* jest kluczowa w całej dalszej pracy.

3.7 Typowe pomyłki w pierwszym programie

Na początku najczęściej trafiają się cztery rodzaje błędów:

- brak średnika ;,
- brak zamykającego nawiasu klamrowego,
- brak `moon(Main);`,
- wpisanie wartości dynamicznej bez nawiasów klamrowych w `content`.

Przykładowo, jeśli napiszesz:

```
1 content {
2   <h1>title</h1>
3 };
```

Listing 3.5: Zły zapis wartości dynamicznej

na stronie zobaczysz dosłownie słowo `title`, a nie wartość zmiennej. Aby wstawić zawartość zmiennej, potrzebny jest zapis `{title}`.

3.8 Co warto zapamiętać po tym rozdziale

Po pierwszym programie powinna zostać Ci w głowie bardzo prosta mapa:

1. `chunk` grupuje logikę,
2. `page` opisuje generowaną stronę,
3. `content` zawiera HTML,
4. `moon(...)` uruchamia wybrany punkt wejścia,
5. `output` wskazuje katalog wynikowy.

Na tych pięciu elementach zbudujemy wszystko, co pojawi się dalej.

Rozdział 4

Struktura programu

4.1 Dlaczego struktura jest wazna

W małym pliku wszystko wydaje się oczywiste. Problem zaczyna się wtedy, gdy projekt ma kilka stron, wspólne metadata, importy, funkcje pomocnicze i powtarzalne fragmenty. Właśnie wtedy porządna struktura przestaje być luksusem, a zaczyna być koniecznością.

MoonChunk daje do tego kilka prostych narzędzi:

- `chunk` jako główny blok organizacyjny,
- `import` do ładowania rzeczy z innych plików,
- `@include` do włączania chunków do chunków,
- `moon(...)` do wybierania punktu wejścia.

4.2 Co może znajdować się w pliku `.mncnk`

Plik MoonChunk zwykle składa się z dwóch części:

1. opcjonalnych `import`-ów na samej górze,
2. jednego lub wielu `chunk`-ów oraz wywołan `moon(...)`.

Przykład:

```
1 import { Shared } from "../shared.mncnk";  
2 import * as Pages from "../pages.mncnk";  
3
```

```
4 chunk "Main" {  
5     @include Shared;  
6     output: "./dist";  
7 };  
8  
9 moon(Main);
```

Listing 4.1: Typowy szkielet pliku MoonChunk

Warto od razu zauważyć dwie rzeczy:

- importy stoja na samej gorze pliku,
- sam import nie uruchamia jeszcze generowania strony.

4.3 Czym jest chunk

chunk to podstawowa jednostka organizacyjna programu. Można go porównać do modułu, ale z naciskiem na wykonanie. Chunk może zawierać:

- metadata,
- deklaracje zmiennych,
- funkcje,
- pętle i warunki,
- strony `page`,
- wyłączenie innych chunków przez `@include`.

Przykład:

```
1 chunk "Main" {  
2     output: "./dist";  
3     title: "Strona glowna";  
4  
5     function greet(name: string): string {  
6         return "Czesc, " + name;  
7     }  
8  
9     page "" {  
10        let userName = "Alicja";
```

```
11   content {
12     <h1>{title}</h1>
13     <p>{greet(userName)}</p>
14   };
15 };
16 };
```

Listing 4.2: Chunk zawierający kilka rodzajów elementów

4.4 Jedna nazwa, jedna odpowiedzialność

Dobrze jest nadawać chunkom nazwy, które odpowiadają ich roli. Przykładowo:

- "Main" – główny punkt wejścia,
- "Metadata" – wspólne metadane,
- "BlogPages" – generowanie podstron bloga,
- "ProductPages" – generowanie kart produktowych.

Taka konwencja bardzo pomaga przy importach i przy pracy zespołowej.

4.5 Wywołanie `moon(...)` jako punkt startowy

Ważną cechą MoonChunk polega na tym, że chunk istnieje jako opis logiki, ale dopiero `moon(...)` powoduje jego wykonanie.

```
1  chunk "Main" {
2    output: "./dist";
3  };
4
5  moon(Main);
```

Listing 4.3: Jawni punkt wejścia

Możesz też uruchomić kilka punktów wejścia:

```
1  chunk "Home" {
2    output: "./dist";
3    page "" {
4      content {
```



```
5      <h1>Home</h1>
6    };
7  };
8 };
9
10 chunk "About" {
11   output: "../dist";
12   page "about" {
13     content {
14       <h1>About</h1>
15     };
16   };
17 };
18
19 moon(Home);
20 moon>About);
```

Listing 4.4: Wiele wywołań moon(...)

Oznacza to, że jeden plik może wygenerować wiele niezależnych zestawów stron.

4.6 Import a wykonanie

To rozróżnienie jest krytyczne. **Import nie znaczy wykonanie**. Import sprawia tylko, że jakiś zasób staje się dostępny w pliku. Dopiero późniejsze `@include` lub `moon(...)` powoduje realne użycie danego fragmentu.

To zachowanie chroni przed nieprzewidywalnym uruchamianiem kodu z plików pomocniczych. Dzięki temu możesz mieć bibliotekę chunków i korzystać z niej tylko tam, gdzie chcesz.

4.7 Eksportowanie chunków

Jeśli chcesz importować chunk z innego pliku, taki chunk powinien być oznaczony jako **export**. Praktycznie wygląda to tak:

```
1 export chunk "Shared" {
2   const siteName: string = "MoonChunk Docs";
3 };
```

Listing 4.5: Chunk eksportowany do innych plików

Brak eksportu oznacza, że dany chunk jest lokalny dla pliku i nie powinien być traktowany jak element publicznego interfejsu.

4.8 Kolejność w pliku

W MoonChunk kolejność ma znaczenie organizacyjne i semantyczne. Najbardziej bezpieczny wzorzec wygląda tak:

1. importy,
2. deklaracje chunków,
3. wywołania `moon(...)`.

Wewnątrz samego chunk-a dobrym zwyczajem jest trzymanie się następującego układu:

1. `@include`,
2. `output` i `metadata`,
3. `env` z globalami,
4. funkcje i stałe,
5. logika stron, pętle, warunki, `page`.

Ten porządek nie jest przypadkowy. Pozwala szybko rozpoznać, co jest wspólna konfiguracja, co jest logika pomocnicza, a co odpowiada za faktyczne generowanie stron.

4.9 Plik główny i pliki pomocnicze

W mniejszych projektach wszystko może mieścić się w jednym pliku. W większych dużo lepiej sprawdza się podział na role, np.:

```
1 content/  
2   site.mcnk  
3   shared.mcnk  
4   blog.mcnk  
5   pages/  
6     home.mcnk  
7     docs.mcnk
```

```
8 data/  
9   posts.json
```

Listing 4.6: Przykładowy podział projektu na pliki

Wtedy:

- `site.mncnk` może być głównym plikiem startowym,
- `shared.mncnk` może przechowywać wspólne metadane i funkcje,
- pliki w `pages/` mogą zawierać chunki dla poszczególnych obszarów witryny,
- katalog `data/` może trzymać zewnętrzne dane do ładowania przez `data(...)`.

4.10 Myslenie warstwowe

Dobra struktura MoonChunk zwykle rozdziela projekt na trzy warstwy:

- **warstwa konfiguracji** – output, metadane, globalne wartości,
- **warstwa logiki** – funkcje, pętle, warunki, obliczenia,
- **warstwa prezentacji** – bloki `content` i HTML.

Im wyraźniej rozdzielisz te warstwy, tym przyjemniej będzie utrzymywać kod.

4.11 Jak rozpoznać, że struktura robi się zła

Oto kilka sygnałów ostrzegawczych:

- jeden plik ma kilkadziesiąt linii i robi wszystko naraz,
- nie wiadomo, który `chunk` jest punktem wejścia,
- metadane są kopiowane do wielu miejsc,
- te same funkcje pojawiają się w kilku plikach,
- importy są przypadkowe i trudno śledzić zależności,
- chunki mają ogólne nazwy typu `"Test"`, `"Temp"`, `"X"`.

Jeśli widzisz takie objawy, warto zatrzymać się na chwilę i przebudować układ programu. MoonChunk dużo zyskuje na czytelnej strukturze.

Rozdział 5

Strony i HTML

5.1 Strona jako finalny produkt

Najważniejszym wynikiem pracy w MoonChunk są strony HTML. W praktyce oznacza to, że ostatecznym celem programu jest stworzenie jednego albo wielu dokumentów, które można otworzyć w przeglądarce, opublikować na serwerze albo dołączyć do większego statycznego serwisu.

W MoonChunk pojedyncza strona powstaje przez instrukcję `page`.

```
1  page "" {  
2    content {  
3      <h1>Witaj</h1>  
4    };  
5  };
```

Listing 5.1: Najprostsza strona

5.2 Jak rozumiec trase strony

Argument przekazywany do `page` określa trasę, a w konsekwencji także nazwę pliku wynikowego. Warto zapamiętać cztery podstawowe przypadki:

- `page ""` generuje `index.html`,
- `page "/"` również generuje `index.html`,
- `page "about"` generuje `about.html`,
- `page "docs/getting-started"` generuje `docs/getting-started.html`.

MoonChunk pomaga tutaj użytkownikowi. Nie musisz dopisywać rozszerzenia `.html` a ani wiodącego ukośnika. Język sam zamienia trasę na odpowiednią ścieżkę pliku.

5.3 Po co to ułatwienie

Dzięki temu zapis tras jest czystszy i bardziej zbliżony do myślenia o strukturze strony, a nie o niskopoziomowej strukturze plików. Kiedy piszesz:

```
1 page "blog/moj-pierwszy-wpis" {  
2   content {  
3     <h1>Moj pierwszy wpis</h1>  
4   };  
5 };
```

Listing 5.2: Trasa w stylu logicznym

bardziej myślisz o adresie logicznym strony niż o tym, czy trzeba dopisać `.html`. To drobiazg, ale przy dużej liczbie podstron daje bardzo wygodny efekt.

5.4 Blok content

Wewnątrz `page` najważniejszym miejscem jest `content`. To tutaj piszesz właściwy HTML strony. MoonChunk traktuje ten blok jako dynamiczny fragment treści, który zostanie wstawiony do wewnętrznego szablonu dokumentu.

```
1 page "" {  
2   content {  
3     <section>  
4       <h1>Dokumentacja</h1>  
5       <p>Opis projektu.</p>  
6     </section>  
7   };  
8 };
```

Listing 5.3: Zawartość strony w bloku content

Ważna zasada brzmi: **blok content wolno umieszczać tylko w kontekście strony**. Nie jest to ogólny blok do wstawiania HTML gdziekolwiek. Jego rola jest konkretna: budowanie treści finalnej strony.

5.5 Dynamiczne wstawianie wartosci

Jedna z najmocniejszych cech MoonChunk polega na tym, że wewnątrz `content` można wstawiać wartości zmiennych i wyniki wyrażen. Służą do tego nawiasy klamrowe:

```
1 page "" {
2   let title = "MoonChunk";
3   let subtitle = "Język do generowania HTML";
4
5   content {
6     <h1>{title}</h1>
7     <p>{subtitle}</p>
8   };
9 };
```

Listing 5.4: Interpolacja wartości w HTML

MoonChunk oblicza wartości w nawiasach i zamienia je na tekst. To bardzo wygodny sposób na tworzenie dynamicznych nagłówków, opisów, list czy sekcji.

5.6 Atrybuty HTML

MoonChunk pozwala także budować dynamiczne atrybuty. Przykład:

```
1 page "" {
2   let slug = "kontakt";
3   let label = "Przejdź do kontaktu";
4
5   content {
6     <a href={"/" + slug}>{label}</a>
7   };
8 };
```

Listing 5.5: Dynamiczny atrybut HTML

W takim zapisie wartość atrybutu `href` jest obliczana dynamicznie. To przydatne zwłaszcza wtedy, gdy generujesz wiele podobnych linków albo sekcji na podstawie danych.

5.7 Czy w `content` można mieszać HTML i logikę

Tak, ale z umiarem. MoonChunk umożliwia kilka stylów pracy.

Styl prosty

Przy małych stronach logika jest niewielka, a w `content` od razu wstawia się wartości:

```
1 page "" {
2   let title = "Start";
3   content {
4     <h1>{title}</h1>
5   };
6 };
```

Listing 5.6: Prosty styl budowy strony

Styl uporządkowany

Przy większej logice lepiej najpierw przygotować dane, a dopiero potem użyć ich w HTML-u:

```
1 page "" {
2   let title = "Oferta";
3   let intro = "Sprawdź nasze usługi";
4
5   content {
6     <header>
7       <h1>{title}</h1>
8       <p>{intro}</p>
9     </header>
10  };
11 };
```

Listing 5.7: Najpierw logika, potem HTML

Drugi styl jest zwykle czytelniejszy w rozbudowanych projektach.

5.8 Petle i warunki wewnątrz strony

W MoonChunk treść strony może być składana warunkowo i iteracyjnie. Oznacza to, że w ramach `page` można używać `if`, `for` i `while`, a wewnątrz nich umieszczać `content`. To bardzo mocny mechanizm.

```
1 page "" {
2   let isLoggedIn: bool = true;
3
4   if (isLoggedIn) {
```

```
5     content {
6         <p>Witaj ponownie.</p>
7     };
8 } else {
9     content {
10        <p>Zaloguj sie, aby zobaczyc wiecej.</p>
11    };
12 };
13 };
```

Listing 5.8: Content budowany warunkowo

To znaczy, że HTML nie musi być jedną, sztywną bryłą. Może być składany fragment po fragmencie w zależności od danych i warunków.

5.9 Więcej niż jeden blok content

Strona może składać się z wielu bloków `content`. Ich zawartość zostanie połączona w takiej kolejności, w jakiej zostały wykonane.

```
1 page "" {
2     content {
3         <header><h1>Sklep</h1></header>
4     };
5
6     content {
7         <section><p>Promocje dnia.</p></section>
8     };
9
10    content {
11        <footer><small>Koniec strony</small></footer>
12    };
13 };
```

Listing 5.9: Kilka bloków content na jednej stronie

To zachowanie bywa bardzo wygodne przy składaniu strony z warunkowych fragmentów, sekcji generowanych w petli albo części importowanych z innych chunków.

5.10 HTML jako wynik, nie jako cel sam w sobie

Warto pamiętać, że MoonChunk nie próbuje zastąpić całego ekosystemu frontendu. Jego zadaniem jest przygotowanie poprawnego i czytelnego HTML-a. To oznacza, że:

- style CSS możesz dołączyć przez metadata,
- skrypty JavaScript możesz dołączyć przez metadata,
- sama treść strony jest najważniejsza w `content`,
- MoonChunk koncentruje się na generowaniu dokumentu, a nie na runtime w przeglądarce.

5.11 Praktyczna rada dotycząca czytelności

Jeśli blok `content` robi się bardzo długi, stosuj zwykłe zasady porządku znane z HTML-a:

- wcięcia,
- logiczne grupowanie sekcji,
- sensowne nazwy klas,
- nie mieszaj zbyt wielu obliczeń bezpośrednio w nawiasach klamrowych.

Zamiast pisać bardzo rozbudowane wyrażenie w środku znacznika, często lepiej najpierw obliczyć wartość w zmiennej, a potem spokojnie jej użyć w HTML-u.

Rozdział 6

Metadata strony

6.1 Po co w ogole sa metadata

Kiedy otwierasz strone w przegladarence, widzisz glownie to, co trafia do sekcji widocznej dla uzytkownika. Ale dokument HTML ma tez `<head>`, a w nim informacje, ktore nie zawsze sa od razu widoczne, ale maja ogromne znaczenie praktyczne. Sa to m.in.:

- tytul strony na karcie przegladarki,
- opis dla wyszukiwarek,
- wskazowki dla robotow indeksujacych,
- dane dla social media,
- linki do arkuszy stylow, ikon i manifestu,
- skrypty dolaczane do dokumentu.

MoonChunk pozwala definiowac te informacje bezposrednio w kodzie `.mncnk`, bez koniecznosci edytowania zewnetrznego szablonu HTML.

6.2 Podstawowa skladnia metadata

Metadata zapisuje sie jako proste instrukcje typu `nazwa: wartosc`; . Przyklad:

```
1 chunk "Main" {  
2   output: "./dist";  
3   title: "MoonChunk Docs";  
4   metaDescription: "Dokumentacja uzytkownika MoonChunk.";
```

```
5
6  page "" {
7      content {
8          <h1>MoonChunk</h1>
9      };
10 };
11 };
12
13 moon(Main);
```

Listing 6.1: Najprostsze metadata strony

W tym przykładzie `title` ustawia tytuł dokumentu, a `metaDescription` opis.

6.3 Dlaczego to rozwiązanie jest wygodne

W klasycznym podejściu trzeba byłoby otwierać szablon HTML i ręcznie wstawiać wiele znaczników do `<head>`. MoonChunk robi to za Ciebie. Ty pracujesz na poziomie intencji: *jaki ma być tytuł, jaki ma być opis, jaka ma być ikona*, a język wstawia to we właściwe miejsca wewnętrznego szablonu dokumentu.

To sprawia, że:

- pliki `.mncnk` stają się głównym miejscem konfiguracji strony,
- łatwiej utrzymać spójność metadata,
- prościej jest nadpisać tylko pojedyncze pola na konkretnej stronie.

6.4 Metadata wspólne i lokalne

Bardzo częsty scenariusz wygląda tak:

- kilka pól metadata jest wspólnych dla całego serwisu,
- wybrane pola, np. `title` albo `metaDescription`, różnią się per strona.

MoonChunk obsługuje taki model bardzo naturalnie. Możesz utworzyć osobny chunk z metadata wspólnymi, a potem dołączyć go przez `@include`.

```
1 export chunk "Metadata" {
2   lang: "pl";
3   charset: "utf-8";
4   viewport: "width=device-width, initial-scale=1";
5   metaAuthor: "MoonChunk Team";
6   metaRobots: "index,follow";
7   themeColor: "#0f172a";
8 };
```

Listing 6.2: Wspólny chunk z metadata

A następnie:

```
1 chunk "DocsHome" {
2   @include Metadata;
3   output: "./dist";
4   title: "Dokumentacja MoonChunk";
5   metaDescription: "Strona główna dokumentacji.";
6
7   page "" {
8     content {
9       <h1>Dokumentacja</h1>
10    };
11  };
12 };
```

Listing 6.3: Nadpisywanie wybranych metadata w konkretnym chunku

Tutaj wspólne pola pochodzą z `Metadata`, a pola lokalne doprecyzowują daną stronę. W praktyce ostatnia ustawiona wartość metadata wygrywa.

6.5 Kategorie wspieranych metadata

MoonChunk wspiera szeroki zestaw pól. Najwygodniej myśleć o nich w grupach.

Metadata dokumentu i języka

To pola opisujące podstawowy kontekst strony:

- `lang`,
- `dir`,

- `htmlClass`,
- `charset`,
- `viewport`.

SEO i informacje podstawowe

To pola najczęściej wykorzystywane przez wyszukiwarki:

- `title`,
- `metaDescription`,
- `metaKeywords`,
- `metaAuthor`,
- `metaRobots`,
- `themeColor`.

Linki zasobow

To pola odpowiadające za zewnętrzne zasoby dokumentu:

- `canonicalUrl`,
- `faviconHref`,
- `appleTouchIconHref`,
- `manifestHref`,
- `preloadLinks`,
- `preconnectLinks`,
- `styles`,
- `headScripts`.

Open Graph i Twitter

Te pola są przydatne przy udostępnianiu stron w social media:

- `ogType`, `ogTitle`, `ogDescription`, `ogImage`, `ogUrl`, `ogSiteName`, `ogLocale`,
- `twitterCard`, `twitterSite`, `twitterCreator`, `twitterTitle`, `twitterDescription`, `twitterImage`.

Elementy struktury całego dokumentu

MoonChunk obsługuje także pola, które trafiają do ciała strony poza samym `content`:

- `bodyClass`,
- `pageId`,
- `topBar`,
- `header`,
- `footer`,
- `modals`,
- `scripts`,
- `bodyEndExtra`.

6.6 Co się dzieje, gdy pole jest puste

MoonChunk ma wewnętrzny szablon bazowy dokumentu. Gdy niektóre opcjonalne pola metadata pozostaną puste, odpowiadające im puste znaczniki nie muszą zostać zachowane w finalnym HTML-u. To ważne, bo dzięki temu wynikowy dokument nie jest zasmiecany pustymi `meta`, `link` albo pustym `title`.

W praktyce warto jednak myśleć tak: jeśli dana informacja ma znaczenie dla strony, uzupełnij ją świadomie. Jeśli nie ma znaczenia, możesz pozostawić pole puste.

6.7 Metadata a treść strony

Częsta pomyłka początkujących polega na mieszaniu tych dwóch warstw. Na przykład:

- `title` nie zastępuje widocznego nagłówka `<h1>`,
- `metaDescription` nie zastępuje pierwszego akapitu na stronie,
- `styles` nie zastępuje samego pliku CSS.

Innymi słowy:

- metadata opisują dokument i jego otoczenie,
- `content` opisuje główną treść widoczną dla użytkownika.

Dobra strona zwykle potrzebuje jednego i drugiego.

6.8 Przykład rozsądnego zestawu startowego

Dla wielu projektów sensowny zestaw bazowy może wyglądać tak:

```
1  chunk "Main" {
2    output: "./dist";
3    lang: "pl";
4    charset: "utf-8";
5    viewport: "width=device-width, initial-scale=1";
6    title: "MoonChunk";
7    metaDescription: "Przykład strony wygenerowanej w MoonChunk.";
8    metaAuthor: "Twoje Imie";
9    metaRobots: "index,follow";
10   themeColor: "#111827";
11   styles: "<link rel='stylesheet' href='/assets/app.css'>";
12
13   page "" {
14     content {
15       <h1>MoonChunk</h1>
16       <p>Start projektu.</p>
17     };
18   };
19 };
```

Listing 6.4: Rozsądny zestaw metadata startowych

To już daje porządną punkt wyjścia do rozwijania serwisu.

6.9 Kiedy wydzielić metadata do osobnego chunku

Jeśli kopiujesz te same pola do kilku miejsc, to prawie zawsze sygnał, że warto wydzielić wspólny **chunk** z metadata. Najczęściej robi się to wtedy, gdy:

- masz więcej niż jedną stronę,
- każda strona ma ten sam **lang**, **charset**, **viewport**,
- wspólne są ikony, skrypty i style,
- tylko tytuł i opis zmieniają się lokalnie.

To jeden z najprostszych sposobów na zachowanie porządku w projekcie.

6.10 O czym warto pamiętać na koniec

Metadata nie są dodatkiem kosmetycznym. Wpływają na:

- wygląd strony w przeglądarce,
- SEO,
- udostępnianie w mediach społecznościowych,
- odbiór strony na urządzeniach mobilnych,
- dołączanie zasobów takich jak CSS, favicon czy skrypty.

W dobrze zaprojektowanym projekcie MoonChunk metadata są traktowane jako element pierwszej klasy, a nie jako coś dopisywanego na końcu.

Rozdział 7

Zmienne, stale i typy

7.1 Po co istnieją zmienne

Strona internetowa rzadko składa się wyłącznie ze stałych, niezmiennych napisów. Zwykle chcemy przechowywać dane takie jak:

- tytuł strony,
- licznik wpisów,
- informacja, czy element ma być pokazany,
- lista kart produktowych,
- obiekt z danymi autora albo wpisu.

Do tego właśnie służą zmienne. Zmienna to **nazwa wskazująca na wartość**. W MoonChunk można pracować zarówno ze zmiennymi zmiennymi, jak i z wartościami, które po nadaniu powinny pozostać stałe.

7.2 `let` i `const`

MoonChunk, podobnie jak współczesny JavaScript, rozróżnia dwa podstawowe sposoby deklarowania lokalnych nazw.

`let`

`let` służy do tworzenia zmiennej, której wartość można później zmienić.

```
1 let counter: int = 0;  
2 counter = counter + 1;
```

Listing 7.1: Zmienna deklarowana przez `let`

`const`

`const` służy do tworzenia stałych, czyli nazwy, której nie wolno później przypisać nowej wartości.

```
1 const title: string = "MoonChunk";
```

Listing 7.2: Stała deklarowana przez `const`

Jeśli potem spróbowałibyśmy zrobić:

```
1 title = "Nowy tytuł";
```

Listing 7.3: Niedozwolona zmiana wartości stałej

MoonChunk zgłosi błąd.

7.3 Czy `let` musi mieć wartość od razu

Nie zawsze. Jedną z praktycznych cech MoonChunk polega na tym, że `let` może zostać zadeklarowany bez wartości początkowej.

```
1 let later: int;  
2 later = 7;
```

Listing 7.4: Deklaracja `let` bez inicjalizacji

To bywa wygodne, gdy:

- jeszcze nie znasz wartości w chwili deklaracji,
- wartość zależy od warunku,
- chcesz najpierw zadeklarować nazwę, a później obliczyć wynik.

Trzeba jednak uważać: **taka zmienna musi zostać zainicjalizowana przed użyciem**. Jeśli spróbowałibyśmy odczytać ją za wcześnie, MoonChunk zgłosi błąd runtime.

7.4 Czy `const` może być puste na starcie

Nie. `const` musi od razu dostać konkretną wartość.

```
1  const maxItems: int = 10;
```

Listing 7.5: Poprawna deklaracja `const`

Taka zasada ma sens: skoro nazwa ma być stała, od razu trzeba wiedzieć, jaką wartość ma reprezentować.

7.5 Typy w MoonChunk

Typ określa, jakiego rodzaju jest dana wartość. To bardzo ważne, bo interpreter musi wiedzieć, czy pracuje z liczbą, tekstem, wartością logiczną, tablicą czy obiektem.

MoonChunk obsługuje następujące typy podstawowe i specjalne:

- `int`,
- `float`,
- `double`,
- `number`,
- `bool`,
- `string`,
- `array`,
- `object`,
- `dict`,
- `null`,
- `undefined`,
- `unknown`,
- `any`,
- `void` dla funkcji nic nie zwracających.

7.6 Typy liczbowe

MoonChunk rozróżnia kilka wariantów liczb, bo w praktyce nie każda liczba ma zachowywać się identycznie.

int

To liczba całkowita. Gdy pracujesz na samych **int**, szczególnie przy dzieleniu, warto pamiętać, że wynik powinien pozostać liczbą całkowitą.

```
1 let a: int = 1 / 2;
```

Listing 7.6: Dzielenie liczb typu **int**

Taki zapis daje wynik 0, a nie 0.5.

float i double

To typy liczb zmiennoprzecinkowych, przydatne wtedy, gdy chcesz zachować część ułamkową.

```
1 let b: double = 1 / 2.0;  
2 let c: float = 1f / 2;
```

Listing 7.7: Dzielenie z częścią ułamkową

Tutaj wynik zachowuje informacje o ułamku.

number

Możesz traktować **number** jako bardziej ogólną kategorię liczby. Przydaje się tam, gdzie nie chcesz przywiązywać się do jednego konkretnego wariantu liczbowego, ale nadal pracujesz na liczbach.

7.7 Typ **bool**

bool przechowuje wartość logiczną: **true** albo **false**. To typ idealny do:

- warunków **if**,
- sterowania pętlami,

- flag typu *czy pokazac element*,
- porównan i sprawdzen.

```
1 let isPublished: bool = true;
```

Listing 7.8: Przykład zmiennej logicznej

7.8 Typ string

`string` oznacza tekst. W praktyce to jeden z najczesciej uzywanych typow, bo strony internetowe sa w duzej mierze zbudowane wlasnie z tekstu.

```
1 let heading: string = "Dokumentacja MoonChunk";
```

Listing 7.9: Przykład tekstu

7.9 Typy złożone: array, object, dict

array

Tablica służy do przechowywania kolekcji elementów w określonej kolejności.

```
1 const tags: array = ["dsl", "html", "static-site"];
```

Listing 7.10: Tablica

object

Obiekt służy do przechowywania nazwanych pol.

```
1 const author: object = { name: "Alicja", role: "writer" };
```

Listing 7.11: Obiekt

dict

`dict` to praktycznie słownikowy wariant obiektu. W MoonChunk traktowany jest jako obiekt o strukturze klucz–wartosc. W dokumentacji użytkowej warto myslec o nim jako o obiekcie przeznaczonym szczególnie do przechowywania danych.

```
1 let point: dict = { x: 1, y: 2 };
```

Listing 7.12: Słownik typu dict

7.10 Typy specjalne: null i undefined

Te dwa typy są podobne tylko pozornie.

null

null oznacza zwykle świadomie ustawiony brak konkretnej wartości.

undefined

undefined oznacza raczej brak przypisanej wartości albo stan pusty wynikający z kontekstu.

W praktyce jeśli dopiero zaczynasz, nie musisz nadużywać tych typów. Warto jednak wiedzieć, że istnieją, bo pojawiają się w bardziej zaawansowanych scenariuszach.

7.11 Typy unknown i any

To typy bardziej zaawansowane. Najłatwiej myśleć o nich jak o tymczasowych pudełkach, które mówią interpreterowi: *na chwilę odłóż ściśle sprawdzanie typu*.

unknown

Jest bezpieczniejszy koncepcyjnie. Mówi: *mam jakąś wartość, ale jeszcze nie chce zakładać, czym ona jest*.

any

Jest bardziej permissive. Mówi: *traktuj tę wartość bardzo elastycznie*.

W praktyce te typy są najczęściej widoczne przy bardziej zaawansowanych rzutowaniach, zwłaszcza kiedy chcesz przejść przez zapis typu `as unknown as ...`.

7.12 Typ void

`void` dotyczy przede wszystkim funkcji. Oznacza, że funkcja nie ma zwracać użytecznej wartości.

```
1 function logTitle(title: string): void {  
2     print(title);  
3     return;  
4 }
```

Listing 7.13: Funkcja bez wartości zwrotnej

7.13 Deklaracja typu jawnie i niejawnie

W wielu miejscach MoonChunk pozwala pominąć jawny typ i oprzeć się na wartości po prawej stronie przypisania.

```
1 let title = "MoonChunk";
```

Listing 7.14: Deklaracja z typem niejawnym

Mozesz też podać typ jawnie:

```
1 let title: string = "MoonChunk";
```

Listing 7.15: Deklaracja z typem jawnym

Jawne typy są szczególnie przydatne wtedy, gdy:

- chcesz jasno opisać zamiar,
- zależy Ci na czytelności,
- pracujesz z danymi z zewnątrz,
- deklarujesz `let` bez wartości początkowej.

7.14 Zgodność typu przy przypisaniu

Jeśli deklarujesz typ, interpreter pilnuje zgodności. Przykład poprawny:

```
1 let count: int = 5;  
2 count = 8;
```

Listing 7.16: Zgodne przypisanie typu

Przykład niepoprawny:

```
1 let count: int = 5;  
2 count = "osiem";
```

Listing 7.17: Niezgodne przypisanie typu

Tutaj MoonChunk powinien zgłosić błąd typu.

7.15 Globalne zmienne w env

Poza zwykłymi zmiennymi lokalnymi MoonChunk wspiera też globalne deklaracje umieszczane w bloku `env`. To przydatne dla wartości wspólnych dla wielu miejsc programu.

```
1 env {  
2   global siteName: string = "MoonChunk Portal";  
3   global pageSize: int = 10;  
4 };
```

Listing 7.18: Globalne wartości w bloku `env`

Ważna zasada: global deklarowany w `env` powinien być od razu zainicjalizowany. Dzięki temu jest przewidywalny i gotowy do użycia wszędzie tam, gdzie ma być widoczny.

7.16 Praktyczne wskazówki dotyczące typów

- Używaj `int`, gdy wynik ma być liczbą całkowitą.
- Używaj `double` lub `float`, gdy spodziewasz się ułamków.
- Używaj `bool` do warunków, a nie liczb zastępujących prawdę i fałsz.
- Używaj `object` lub `dict` do grupowania nazwanych danych.
- Używaj `array` do list i kolekcji.
- Po `unknown` i `any` sięgaj dopiero wtedy, gdy zwykłe typy nie wystarczają.

7.17 Najczęstsze błędy początkujących

- traktowanie `const` jak `let`,
- odczytywanie `let` przed inicjalizacją,

- mieszanie typu liczbowego z tekstem bez świadomego rzutowania,
- wpisywanie wartości logicznej tam, gdzie oczekiwana jest liczba,
- kopiowanie tej samej zmiennej w to samo miejsce zamiast wykonania zwykłego przypisania.

W następnym rozdziale przejdziemy od samych typów do wyrażeń, czyli do tego, jak MoonChunk oblicza wartości i łączy je ze sobą.

Rozdział 8

Wyrażenia i operatory

8.1 Czym jest wyrażenie

Wyrażenie to fragment kodu, który po obliczeniu daje jakąś wartość. Na przykład:

- `2 + 3` daje liczbę,
- `title + "!"` daje tekst,
- `views > 100` daje wartość logiczną,
- `isPublished and isFeatured` daje wartość logiczną,
- `condition ? a : b` wybiera jedną z dwóch wartości.

W MoonChunk wyrażenia pojawiają się praktycznie wszędzie: po prawej stronie przypisania, w warunkach, w petlach, w argumentach funkcji, w atrybutach HTML i we wstawieniach do `content`.

8.2 Operatory arytmetyczne

MoonChunk obsługuje podstawowe operacje liczbowe:

- `+` dodawanie,
- `-` odejmowanie,
- `*` mnożenie,
- `/` dzielenie,

- % reszta z dzielenia.

```
1 let sum: int = 2 + 3;
2 let diff: int = 10 - 4;
3 let mul: int = 6 * 7;
4 let div: int = 8 / 2;
5 let mod: int = 7 % 2;
```

Listing 8.1: Podstawowe obliczenia

8.3 Kolejność wykonywania działań

Podobnie jak w matematyce, nie wszystkie operacje są wykonywane od lewej do prawej w tej samej kolejności. Najpierw interpreter bierze pod uwagę priorytety operatorów. Najprostsza intuicja jest taka:

- nawiasy mają najwyższy priorytet,
- potem mnożenie, dzielenie i modulo,
- potem dodawanie i odejmowanie,
- dalej porównania,
- a na końcu bardziej złożone konstrukcje logiczne i warunkowe.

Jeśli chcesz uniknąć wątpliwości, używaj nawiasów.

```
1 let value: int = (2 + 3) * 4;
```

Listing 8.2: Nawiasy zwiększają czytelność

8.4 Dzielenie przez zero i modulo przez zero

MoonChunk traktuje te sytuacje jako błąd. To dobre, bo wynik typu *nieskonczonosc* byłby w praktyce niebezpieczny i mogłoby ukryć pomyłkę w logice programu.

```
1 let a: int = 10 / 0;
2 let b: int = 10 % 0;
```

Listing 8.3: Przykłady niedozwolonych operacji

Takie zapisy powinny zakończyć się komunikatem diagnostycznym.

8.5 Operatory porównania

Porównania są potrzebne wtedy, gdy chcemy sprawdzić relacje między wartościami. MoonChunk obsługuje:

- `==` równość,
- `!=` nierówność,
- `<` mniejsze niż,
- `>` większe niż,
- `<=` mniejsze lub równe,
- `>=` większe lub równe.

```
1 let isZero: bool = count == 0;  
2 let isBig: bool = views > 100;  
3 let isReady: bool = step >= 3;
```

Listing 8.4: Porównania w praktyce

Każde takie porównanie daje wynik typu `bool`.

8.6 Operatory logiczne

Do składania warunków MoonChunk używa operatorów tekstowych:

- `and`,
- `or`,
- `not`.

```
1 let visible: bool = isPublished and not isArchived;  
2 let premium: bool = hasSubscription or hasInvitation;
```

Listing 8.5: Logika w MoonChunk

To podejście jest czytelne i dobrze pasuje do składni zorientowanej na człowieka.

8.7 Operator warunkowy

MoonChunk wspiera operator trójargumentowy, znany z wielu innych języków. Pozwala on wybrać jedną z dwóch wartości na podstawie warunku.

```
1 let label: string = isPublished ? "Opublikowany" : "Szkic";
```

Listing 8.6: Operator warunkowy

To zwięzły zapis przydatny wtedy, gdy chcesz szybko wybrać jedną z dwóch prostych wartości. Jeśli logika robi się bardziej rozbudowana, zwykle lepiej przejść do pełnego `if`.

8.8 Łączenie tekstu

Operator `+` służy nie tylko do dodawania liczb. W praktyce jest też wygodnym sposobem na składanie tekstów.

```
1 let fullTitle: string = "MoonChunk" + " " + "Docs";
```

Listing 8.7: Łączenie tekstu

Z takiego mechanizmu często korzysta się przy budowaniu linków, nagłówków, klas CSS czy fragmentów HTML składanych do jednej zmiennej tekstowej.

8.9 Przypisanie jako wyrażenie i zwykłe przypisanie

MoonChunk pozwala przypisywać nową wartość do już istniejącej zmiennej:

```
1 let counter: int = 0;  
2 counter = counter + 1;
```

Listing 8.8: Zwykłe przypisanie

To nie jest redeklaracja, tylko zmiana wartości istniejącej zmiennej typu `let`. Różnica jest fundamentalna:

- `let counter = 0`; tworzy nową nazwę,
- `counter = counter + 1`; zmienia wartość tej nazwy.

8.10 Operator inkrementacji

MoonChunk wspiera dwa znane zapisy zwiększania wartości o jeden:

- `i++` – postinkrementacja,
- `++i` – preinkrementacja.

```
1 let i: int = 1;  
2 print(i++);  
3 print(++i);
```

Listing 8.9: Preinkrementacja i postinkrementacja

W praktyce:

- `i++` zwraca stara wartość, a dopiero potem zwiększa zmienną,
- `++i` najpierw zwiększa zmienną, a potem zwraca nową wartość.

To drobna różnica, ale przy bardziej zaawansowanej logice bywa istotna.

8.11 Wyrażenia w HTML

W MoonChunk wyrażenia mogą trafiać bezpośrednio do `content`. Przykład:

```
1 content {  
2   <p>{views > 100 ? "Popularny wpis" : "Nowy wpis"}</p>  
3 };
```

Listing 8.10: Wyrażenie wewnątrz content

To bardzo wygodne, ale warto zachować umiar. Jeśli wyrażenie staje się długie, często lepiej obliczyć je najpierw w zmiennej.

8.12 Wyrażenia w indeksowaniu i dostępie do pól

Wyrażenia mogą pojawiać się również przy dostępie do tablic i obiektów.

```
1  const cards = [  
2    { title: "Parser" },  
3    { title: "Runtime" }  
4  ];  
5  
6  let index: int = 1;  
7  let key: string = "title";  
8  print(cards[index][key]);
```

Listing 8.11: Dostęp dynamiczny do danych

To pokazuje, że MoonChunk nie ogranicza się do prostych stałych ścieżek typu `obj.title`. Potrafi też pracować z kluczami i indeksami obliczanymi dynamicznie.

8.13 Kiedy wyrażenie jest zbyt złożone

Jeśli podczas czytania własnego kodu sam musisz zatrzymać się na kilka sekund, aby zrozumieć jedno wyrażenie, to sygnał, że warto je uprościć. Dobra praktyka wygląda tak:

```
1  let canShowBanner: bool = isPublished and not isArchived;  
2  let bannerText: string = canShowBanner ? "Gotowe" : "Ukryte";
```

Listing 8.12: Uproszczenie przez zmienne pomocnicze

Taki zapis jest zwykle czytelniejszy niż jeden bardzo długi warunek wpisany wprost do HTML-a albo do warunku petli.

Rozdział 9

Rzutowanie typów

9.1 Po co w ogóle rzutować typy

Rzutowanie jest potrzebne wtedy, gdy dana wartość ma już jakiś typ, ale w konkretnym miejscu programu chcesz spojrzeć na nią inaczej. Typowe przypadki to:

- zamiana napisu "42" na liczbę,
- zamiana liczby na tekst,
- przejście przez typ `unknown` albo `any`,
- świadome dopasowanie danych z zewnątrz do oczekiwanego kształtu.

MoonChunk wspiera dwa style rzutowania:

- styl z nawiasem typu, np. `(int)value`,
- styl z operatorem `as`, np. `value as int`.

9.2 Rzutowanie w stylu `(typ)wartosc`

To zapis zwarty i bezpośredni.

```
1 let count: int = (int)"42";  
2 let label: string = (string)15;
```

Listing 9.1: Rzutowanie w stylu C

Taki styl jest szczególnie wygodny, gdy chcesz szybko pokazać, jaki ma być efekt konwersji.

9.3 Rzutowanie przez as

Druga mozliwosc to zapis z `as`:

```
1 let count: int = "42" as int;  
2 let text: string = 15 as string;
```

Listing 9.2: Rzutowanie przez as

Ten styl jest bardzo czytelny w dluzszych wyrazeniach, bo przypomina naturalne czytanie: *weź wartosc i potraktuj ja jako dany typ*.

9.4 Rzutowanie liczb i tekstu

Najczestsze i najbardziej intuicyjne konwersje to:

- tekst do liczby,
- liczba do tekstu,
- liczba jednego typu do liczby innego typu,
- bool do liczby,
- liczba do ogolnego typu `number`.

```
1 let a: int = "42" as int;  
2 let b: double = "3.5" as number;  
3 let c: string = b as string;  
4 let d: float = (float)10;
```

Listing 9.3: Czyste rzutowania

9.5 Rzutowanie do bool

To najbardziej specyficzny przypadek w MoonChunk i warto go dobrze zrozumiec. Bezpośredni zapis z `as` do `bool` jest ograniczony.

```
1 let flag: bool = 3 as bool;
```

Listing 9.4: Niedozwolone bezpośrednie rzutowanie przez as

Taki zapis nie powinien byc akceptowany. Jesli chcesz doprowadzic wartosc do `bool`, masz dwie główne drogi.

Droga pierwsza: rzutowanie w stylu `(bool)wartosc`

```
1 let flag: bool = (bool)3;
```

Listing 9.5: Rzutowanie do bool przez zapis nawiasowy

Droga druga: przejście przez `unknown` albo `any`

```
1 let flag: bool = 3 as unknown as bool;
```

Listing 9.6: Most przez `unknown`

Ten zapis mówi w praktyce: *na chwile potraktuj wartość jako niejawna, a potem rzutuj ją do bool*.

9.6 Jak zachowuje się liczba rzutowana do bool

W bieżącej semantyce MoonChunk dodatnie wartości liczbowe po przejściu do `bool` są interpretowane jako `true`, a wartości niedodatnie jako `false`. Dlatego:

```
1 let a: bool = (bool)3;  
2 let b: bool = (bool)0;  
3 let c: bool = (bool)(-3);
```

Listing 9.7: Przykładowe wyniki rzutowania liczby do bool

odpowiednio daje intuicyjnie:

- `a = true`,
- `b = false`,
- `c = false`.

To jedna z tych zasad, które warto po prostu zapamiętać i stosować konsekwentnie.

9.7 Przechodzenie przez `unknown` i `any`

W wielu nowoczesnych systemach typów stosuje się etap pośredni, gdy bezpośrednia konwersja jest za mocna albo zbyt ryzykowna. MoonChunk dopuszcza taki most.

```
1 let visible: bool = (+3) as unknown as bool;  
2 let hidden: bool = (-3) as unknown as bool;
```

Listing 9.8: Lancuch rzutowan

To nie jest zapis dla początkującego na co dzień, ale warto wiedzieć, co oznacza, ponieważ pojawia się w bardziej zaawansowanych przypadkach.

9.8 Rzutowanie obiektów, słowników i tablic

MoonChunk pozwala też sprawdzać i dopasowywać typy złożone.

```
1 let list: array = [1, 2, 3] as array;  
2 let user: object = { name: "Ala" } as object;  
3 let point: dict = { x: 1, y: 2 } as dict;
```

Listing 9.9: Rzutowania typów złożonych

Jeśli jednak spróbowałibysmy potraktować liczbę jak tablicę albo prostą wartość jak obiekt, MoonChunk powinien zgłosić błąd.

9.9 Rzutowanie do null i undefined

Są to przypadki szczególne. Taka konwersja ma sens tylko wtedy, gdy wartość faktycznie jest już `null` albo `undefined`. MoonChunk nie traktuje ich jako magicznych typów, które można dowolnie uzyskać z każdej wartości.

9.10 Co nie jest rzutowaniem

Początkujący czasem mieszają dwa różne pojęcia:

- **rzutowanie** – świadoma zmiana sposobu interpretacji wartości,
- **zwykłe przypisanie** – podstawienie wartości do zmiennej.

To, że zmienna ma typ `bool`, nie znaczy, że można do niej przypisać dowolną liczbę bez konwersji.

```
1 let a: bool = 5 > 5;  
2 a = 3;
```

Listing 9.10: Przypisanie bez zgodności typu

To powinno zakończyć się błędem typu, bo tu nie zaszło żadne prawidłowe rzutowanie.

9.11 Kiedy rzutować, a kiedy lepiej nie

Rzutowanie jest narzędziem bardzo przydatnym, ale nadużywane szybko zamienia kod w coś trudnego do zrozumienia. Dobra praktyka jest prosta:

- rzutuj wtedy, gdy faktycznie przechodzisz między różnymi reprezentacjami danych,
- unikaj rzutowań lancuchowych, jeśli zwykła deklaracja typu wystarczy,
- przy danych zewnętrznych najpierw zrozum, co naprawdę przychodzi,
- nie traktuj `unknown` i `any` jako domyślnego rozwiązania na wszystko.

9.12 Rzutowanie w praktyce strony

Bardzo realistyczny przypadek dotyczy danych z pliku JSON, gdzie niektóre wartości mogą przyjść jako tekst, a Ty chcesz wykonać na nich obliczenia.

```
1 let rawViews: string = "120";  
2 let views: int = rawViews as int;  
3 let text: string = "Wyświetlenia: " + (views as string);
```

Listing 9.11: Konwersja danych zewnętrznych

Dzięki temu jedną wartość można w odpowiednim momencie potraktować jako liczbę, a w innym jako tekst gotowy do pokazania na stronie.

Rozdział 10

Instrukcje warunkowe

10.1 Po co programowi warunki

Strona internetowa rzadko jest absolutnie identyczna w każdej sytuacji. Czasem chcemy:

- pokazać sekcje tylko dla opublikowanych wpisów,
- ukryć komunikat, jeśli lista jest pusta,
- wyświetlić inny tekst dla użytkownika premium,
- nadać inny styl w zależności od stanu danych.

Do takich sytuacji służy instrukcja `if`. Pozwala powiedzieć: *jeśli warunek jest prawdziwy, wykonaj ten blok; w przeciwnym razie wykonaj inny albo nie wykonuj niczego*.

10.2 Najprostsza forma `if`

```
1  if (isPublished) {  
2      print("Wpis jest opublikowany");  
3  };
```

Listing 10.1: Podstawowy warunek `if`

Jeśli `isPublished` ma wartość `true`, instrukcja wewnątrz bloku zostanie wykonana. Jeśli ma wartość `false`, nic się nie stanie.

10.3 Warunek musi być typu `bool`

To jedna z bardzo ważnych zasad MoonChunk. Wyrażenie w nawiasach po `if` powinno po obliczeniu dawać `true` albo `false`. Nie należy traktować liczb albo tekstów jako niejawnych zastępników prawdy.

Poprawnie:

```
1 let showBanner: bool = views > 100;
2 if (showBanner) {
3     print("Pokaż baner");
4 };
```

Listing 10.2: Poprawny warunek logiczny

Niepoprawnie:

```
1 if (3) {
2     print("To powinno być błędem");
3 };
```

Listing 10.3: Niepoprawny warunek niebędący `bool`

10.4 `if` z `else`

Bardzo często potrzebujemy dwóch alternatyw. Wtedy korzystamy z `else`.

```
1 if (isPublished) {
2     print("Status: opublikowany");
3 } else {
4     print("Status: szkic");
5 };
```

Listing 10.4: Pełny warunek `if-else`

To prosty, bardzo czytelny zapis: jedna droga dla prawdy, druga dla fałszu.

10.5 Warunki a HTML

Instrukcje warunkowe są szczególnie przydatne przy generowaniu treści strony. Możesz decydować, który fragment `content` ma zostać dołączony.

```
1  page "" {
2    let hasPosts: bool = true;
3
4    if (hasPosts) {
5      content {
6        <p>Mamy wpisy do pokazania.</p>
7      };
8    } else {
9      content {
10       <p>Brak wpisow.</p>
11     };
12   };
13 }
```

Listing 10.5: Warunkowe generowanie fragmentu strony

To jeden z najważniejszych praktycznych wzorców w MoonChunk.

10.6 Warunki złożone

Prawdziwa siła `if` pojawia się wtedy, gdy warunek składa się z kilku elementów. Na przykład:

```
1  if (isPublished and not isArchived and views > 100) {
2    print("Pokaz promowany wpis");
3  };
```

Listing 10.6: Złożony warunek logiczny

Takie warunki są bardzo przydatne, ale trzeba pilnować czytelności. Jeśli logika staje się długa, warto podzielić ją na zmienne pomocnicze.

```
1  let canPromote: bool = isPublished and not isArchived;
2  let isPopular: bool = views > 100;
3
4  if (canPromote and isPopular) {
5    print("Pokaz promowany wpis");
6  };
```

Listing 10.7: Podział złożonego warunku na kroki

10.7 Zagnieżdzone warunki

Mozna też umieszczać jeden `if` wewnątrz drugiego. Trzeba jednak robić to z umiarem.

```
1  if (isPublished) {  
2    if (views > 100) {  
3      print("Wpis jest opublikowany i popularny");  
4    };  
5  };
```

Listing 10.8: Zagnieżdzone warunki

Zagnieżdżenia są czasem naturalne, ale ich nadużywanie pogarsza czytelność. Często lepszym rozwiązaniem jest obliczenie kilku nazwanych warunków pomocniczych.

10.8 If a zakresy nazw

W MoonChunk ciało instrukcji `if` jest miejscem, w którym trzeba uważać na redefinicje nazw. Język celowo nie zachowuje się tu jak czysty, niezależny blok do bezkarnego zasłaniania tych samych nazw z góry.

Przykład potencjalnie problematyczny:

```
1  const x: int = 3;  
2  
3  if (true) {  
4    const x: int = 5;  
5  };
```

Listing 10.9: Redeklaracja w ciele `if`

Taki zapis należy traktować jako błąd redefinicji. Jeśli chcesz utworzyć odrębny, celowo zagnieżdżony zakres, skorzystaj z jawnego bloku `{ ... }`, o czym będzie mowa w osobnym rozdziale.

10.9 Kiedy używać `if`, a kiedy operatora `?:`

To częste pytanie. Dobra zasada jest bardzo praktyczna:

- operator `?:` jest dobry dla bardzo prostego wyboru jednej z dwóch wartości,
- `if` jest lepszy, gdy trzeba wykonać więcej niż jedną instrukcję albo zbudować HTML.

Przykład prosty:

```
1 let label: string = isPublished ? "Tak" : "Nie";
```

Listing 10.10: Operator warunkowy do malego wyboru

Przykład bardziej rozbudowany:

```
1 if (isPublished) {  
2     print("Wpis gotowy");  
3     content {  
4         <span>Opublikowany</span>  
5     };  
6 } else {  
7     print("Wpis jeszcze nie gotowy");  
8 };
```

Listing 10.11: if do większej logiki

10.10 Typowe zastosowania w stronach

W prawdziwych projektach if jest szczególnie przydatne do:

- pokazywania pustego stanu listy,
- warunkowego renderowania banerów,
- wyboru sekcji dla różnych typów treści,
- nadpisywania wybranych fragmentów na podstawie danych,
- ochrony przed próbą dostępu do danych, których może nie być.

10.11 Najczęstsze błędy przy warunkach

- wpisanie `=` zamiast `==`,
- użycie liczby lub tekstu zamiast wartości logicznej,
- zbyt rozbudowany warunek bez nazw pomocniczych,
- redefiniowanie tej samej nazwy w ciele if,
- zapominanie o średnikach po blokach, jeśli składnia ich wymaga.

Jesli MoonChunk zgłosi blad w warunku, warto najpierw sprawdzic trzy rzeczy:

1. czy porownanie jest zapisane jako `==`, a nie `=`,
2. czy wynik warunku naprawde jest typu `bool`,
3. czy nawiasy i klamry zamykaja sie poprawnie.

Rozdział 11

Petle: powtarzanie pracy bez kopiowania kodu

11.1 Po co istnieją petle

Petla pozwala wykonać podobną operację wiele razy. To jedna z najważniejszych idei programowania. Zamiast ręcznie pisać dziesięć podobnych sekcji HTML albo dziesięć razy zwiększać licznik, można zapisać regułę powtarzania.

Na stronach internetowych petle przydają się np. do:

- generowania list wpisów,
- budowania siatki kart,
- przechodzenia po elementach menu,
- wykonywania operacji na wszystkich rekordach danych,
- tworzenia wielu podstron według jednego schematu.

MoonChunk wspiera dwa główne rodzaje petli: `for` i `while`.

11.2 Petla `for`

Petla `for` jest bardzo dobra wtedy, gdy chcesz wyraźnie kontrolować licznik. W MoonChunk ma ona klasyczny, jawny zapis.

```
1  for (let i: int = 0; i < 5; i++) {  
2    print(i);
```

```
3 };
```

Listing 11.1: Klasyczna petla for

Ten zapis czyta sie tak:

1. utwórz licznik `i` i ustaw go na 0,
2. dopóki `i < 5`, wykonuj ciało petli,
3. po każdym obiegu zwiększ `i` o jeden.

11.3 Budowa petli for krok po kroku

Petla `for` ma trzy główne części wewnątrz nawiasów:

- inicjalizacje,
- warunek,
- aktualizacje.

```
1 for (let i: int = 0; i < 10; i = i + 2) {
2   print(i);
3 };
```

Listing 11.2: Trzy części petli for

Tutaj licznik rośnie co dwa kroki, a nie co jeden.

11.4 Czy aktualizacja w for musi być i++

Nie. MoonChunk pozwala wpisać zwykłe wyrażenie aktualizujące. Dlatego poprawne są oba style:

```
1 for (let i: int = 0; i < 10; i++) {
2   print(i);
3 };
4
5 for (let i: int = 0; i < 10; i = i + 2) {
6   print(i);
7 };
```

Listing 11.3: Dwie formy aktualizacji licznika

To bardzo praktyczne, bo pozwala zapisac nie tylko krok o jeden, ale dowolna logike przesuwania licznika.

11.5 Petla while

`while` przydaje sie wtedy, gdy nie chcesz myslec wprost o klasycznym liczniku albo liczba powtorzen nie jest z gory oczywista. Skladnia jest prosta:

```
1 let counter: int = 0;
2
3 while (counter < 3) {
4     counter = counter + 1;
5 };
```

Listing 11.4: Podstawowa petla while

Petla powtarza swoje ciało tak dlugo, jak warunek pozostaje `true`.

11.6 Warunek petli musi byc typu bool

Podobnie jak przy `if`, MoonChunk wymaga, by warunek w `for` i `while` byl wartoscia logiczna. To oznacza, ze zapis taki jak:

```
1 while (3) {
2     print("blad");
3 };
```

Listing 11.5: Niepoprawny warunek petli

powinien skonczyc sie bledem typu.

11.7 Petle a generowanie HTML

Petle sa szczegolnie ciekawe wtedy, gdy budujesz wiele podobnych fragmentow strony. MoonChunk pozwala umieszczac `content` wewnatrz petli, jesli znajdujesz sie w ciele `page`.

```
1 page "" {
2     const cards = [
3         { title: "A", description: "one", button: "Read more" },
4         { title: "B", description: "two", button: "Read more" },
5         { title: "C", description: "three", button: "Read more" }
```

```
6 ];
7
8 for (let i: int = 0; i < 3; i++) {
9     content {
10         <article class="card">
11             <h3>{cards[i].title}</h3>
12             <p>{cards[i].description}</p>
13             <button>{cards[i].button}</button>
14         </article>
15     };
16 };
17 };
```

Listing 11.6: Content generowany w petli

To jeden z najbardziej naturalnych wzorców pracy z MoonChunk.

11.8 break i continue

W petlach dostępne są dwie bardzo ważne instrukcje sterujące.

break

Natychmiast kończy całą petle.

continue

Pomija resztę bieżącego obiegu i przechodzi do kolejnej iteracji.

```
1 for (let i: int = 0; i < 10; i++) {
2     if (i % 2 != 0) {
3         continue;
4     };
5
6     if (i == 6) {
7         break;
8     };
9
10    print(i);
11 };
```

Listing 11.7: break i continue w praktyce

W tym przykładzie:

- liczby nieparzyste są pomijane,
- cała petla kończy się przy `i == 6`.

11.9 Gdzie wolno używać `break` i `continue`

Tylko wewnątrz petli. To bardzo ważna zasada. Jeśli spróbujesz użyć tych instrukcji poza `for` albo `while`, MoonChunk zgłosi błąd sterowania przepływem.

11.10 Petle zagnieżdzone

MoonChunk pozwala umieszczać jedną petle w drugiej.

```
1  for (let i: int = 1; i <= 3; i++) {
2    for (let j: int = 1; j <= i; j++) {
3      print(i, j);
4    };
5  };
```

Listing 11.8: Petle zagnieżdzone

To przydatne przy generowaniu siatek, tabel i struktur dwuwymiarowych.

11.11 Petle a zakres nazw

Ciało petli w MoonChunk korzysta z dość rygorystycznych zasad redekleracji. W praktyce oznacza to, że nie powinienś traktować `for` ani `while` jak wolnej przestrzeni na ponowne dekladowanie nazw z najbliższego otoczenia.

Jeśli potrzebujesz celowo stworzyć osobny zakres do eksperymentalnej pracy z nazwą, lepiej użyć jawnego bloku `{ ... }` albo wydzielić funkcję.

11.12 Najczęstsze błędy przy petlach

- zapomnienie o zmianie wartości sterującej w `while`,
- warunek niebedący typu `bool`,
- użycie `break` albo `continue` poza petlą,
- błędna aktualizacja licznika,
- redeklaracja tej samej nazwy w nieodpowiednim zakresie,
- nieuważne dzielenie albo modulo przez zero przy logice sterującej petlą.

11.13 Kiedy wybrać `for`, a kiedy `while`

Najprostsza zasada:

- wybierz `for`, gdy masz licznik lub jasny schemat iteracji,
- wybierz `while`, gdy petla ma działać dopóki zachodzi pewien stan.

Jeśli generujesz kolejne karty z tablicy według indeksu, `for` zwykle będzie bardziej naturalny. Jeśli czekasz, aż jakaś wartość osiągnie określony stan, `while` bywa prostszy.

Rozdział 12

Zasiegi, bloki i widoczność nazw

12.1 Dlaczego zasięg jest tak ważny

Zasięg odpowiada na pytanie: *z jakiego miejsca programu dana nazwa jest widoczna i do jakiej wartości się odnosi*. Bez tego bardzo szybko powstałby chaos. Dwie różne części programu mogłyby przypadkiem walczyć o tę samą nazwę i nadpisywać sobie dane.

W MoonChunk temat zasięgów jest szczególnie istotny, bo łączy:

- zwykłe zmienne,
- funkcje,
- ciała warunków i pętli,
- odrębne chunki,
- jawne bloki `{ ... }`,
- specjalny zapis `parent::name`.

12.2 Najprostsza intuicja

Jeśli deklarujesz nazwę w pewnym miejscu, to najpierw należy zakładać, że jest ona widoczna lokalnie w tym miejscu i w tym, co z niego wynika. Jednak nie każdy blok w MoonChunk daje identyczną swobodę cieniowania nazw.

To celowy projekt: język stawia na przewidywalność i chce ograniczać przypadkowe kolizje nazw.

12.3 Redeclaracja a zwykle przypisanie

Na początek trzeba odróżnić dwie rzeczy:

- **redeklaracja** tworzy nową nazwę o tej samej nazwie,
- **przypisanie** zmienia wartość już istniejącej nazwy typu `let`.

Przykład poprawnego przypisania:

```
1 let x: int = 1;  
2 x = x + 1;
```

Listing 12.1: Zmiana wartości przez przypisanie

Przykład błędnej redeklaracji w tym samym zakresie:

```
1 let x: int = 1;  
2 let x: int = 2;
```

Listing 12.2: Błędna redeklaracja w tym samym miejscu

12.4 Granice, przy których ta sama nazwa jest dozwolona

MoonChunk dopuszcza powtórzenie tej samej nazwy tylko w wybranych, świadomych granicach. Najważniejsze z nich to:

- osobny `chunk`,
- osobne wywołanie funkcji,
- jawny blok `{ ... }`.

Oznacza to, że ten sam identyfikator może oznaczać coś innego w innym chunku albo wewnątrz funkcji, bez konfliktu z nazwą z zewnątrz.

12.5 Osobne chunki jako osobne granice

Dwa różne chunki mogą korzystać z tej samej nazwy i nie wchodzi sobie w drogę.

```
1 chunk "First" {
2   const x: int = 1;
3   print("First.x:", x);
4 };
5
6 chunk "Second" {
7   const x: int = 2;
8   print("Second.x:", x);
9 };
```

Listing 12.3: Ta sama nazwa w dwóch chunkach

To naturalne: każdy chunk jest osobną jednostką wykonania.

12.6 Funkcja jako granica zasięgu

Funkcje również tworzą istotną granicę. Przykład:

```
1 chunk "Main" {
2   const x: int = 3;
3
4   function demo(): int {
5     const x: int = 5;
6     return x;
7   }
8
9   print("outer:", x);
10  print("inner:", demo());
11 };
```

Listing 12.4: Ta sama nazwa na zewnątrz i wewnątrz funkcji

Tutaj wewnętrzne `x` funkcji nie niszczy zewnętrznego `x`. To oddzielna przestrzeń. Dodatkowo przy rekurencji każde kolejne wywołanie funkcji ma swój własny zestaw lokalnych zmiennych.

12.7 Jawny blok { ... }

MoonChunk wspiera też samodzielny, jawny blok, który pozwala świadomie utworzyć nowy zagnieżdżony zakres.

```
1 let i: int = 1;
```

```
2
3 {
4   let i: int = 99;
5   print(i);
6 };
7
8 print(i);
```

Listing 12.5: Jawny blok jako osobny zakres

W takim przypadku wewnętrzne `i` jest osobna nazwa dla wnętrza bloku, a zewnętrzne `i` pozostaje bez zmian.

12.8 Dlaczego to rozwiązanie jest przydatne

Jawny blok pozwala:

- ograniczyć życie zmiennych do małego fragmentu logiki,
- eksperymentować z nazwami pomocniczymi bez zanieczyszczania otoczenia,
- budować czytelniejsze etapy obliczeń,
- tworzyć bezpieczne, lokalne konteksty pracy.

To bardzo przyjemny mechanizm wszędzie tam, gdzie funkcja byłaby zbyt dużą konstrukcją, a zwykły ciąg instrukcji – zbyt spłaszczony.

12.9 Ciała `if`, `for` i `while` są bardziej rygorystyczne

To ważna cecha MoonChunk. Chociaż instrukcje warunkowe i pętle korzystają z bloków, nie należy traktować ich jak pełnej, swobodnej przestrzeni do ponownego deklarowania tej samej nazwy z najbliższego otoczenia.

Przykład problematyczny:

```
1  const x: int = 3;
2
3  if (true) {
4    const x: int = 5;
5  };
```

Listing 12.6: Redeclaracja w ciele `if`

Ten wzorzec powinien być traktowany jako błąd redekłaracji. Jeśli celem jest świadome wydzielenie osobnego zasięgu z tą samą nazwą, lepiej użyć jawnego bloku `{ ... }` albo funkcji.

12.10 Odczyt wartości z nadrzędnego zakresu przez `parent::`

MoonChunk daje też specjalny zapis pozwalający sięgnąć do wyższej warstwy zasięgu. Przykład:

```
1  let i: int = 1;
2
3  {
4      let i: int = 99;
5      {
6          let fromParent1: int = parent::i;
7          let fromParent2: int = parent::parent::i;
8          print(fromParent1);
9          print(fromParent2);
10     };
11 };
```

Listing 12.7: Odczyt z `parent::`

Tutaj:

- `parent::i` oznacza `i` z bezpośrednio wyższego poziomu,
- `parent::parent::i` oznacza `i` dwa poziomy wyżej.

To narzędzie zaawansowane, ale bardzo przydatne, gdy świadomie pracujesz na zagnieżdżonych blokach.

12.11 Kiedy nie używać `parent::`

Jeśli można przekazać wartość wprost do funkcji albo zbudować kod czytelniej przez zmienne pomocnicze, zwykle warto wybrać prostsze rozwiązanie. `parent::` jest przydatny, ale nadużywany szybko utrudnia śledzenie przepływu danych.

12.12 Globalne wartości a zasięgi lokalne

Zmienne globalne z bloku `env` są osobnym mechanizmem. Służą do wspólnych wartości, ale nie powinny być traktowane jako substytut dobrej organizacji lokalnych nazw.

Praktyczna rada jest prosta:

- rzeczy naprawdę wspólne wrzucaj do globali,
- rzeczy lokalne zostaw lokalnie,
- nie buduj wszystkiego na globalach tylko dlatego, że są wygodne.

12.13 Najczęstsze błędy związane z zasięgami

- redeklaracja tej samej nazwy w tym samym miejscu,
- probowanie cieniowania nazwy w `if`, `for` lub `while`,
- odczyt zmiennej przed inicjalizacją,
- zbyt głębokie poleganie na `parent::`,
- mieszanie odpowiedzialności globali i zwykłych zmiennych lokalnych.

12.14 Złota zasada czytelności

Jeśli po kilku dniach nie jesteś w stanie szybko odpowiedzieć, skąd bierze się wartość jakiejś nazwy, to znaczy, że struktura zasięgów jest zbyt skomplikowana. Dobrze napisany kod MoonChunk ma być czytelny nie tylko dla interpretera, ale też dla człowieka.

Rozdział 13

Funkcje i rekurencja

13.1 Po co istnieją funkcje

Funkcja pozwala nazwać pewien fragment logiki i wywoływać go wiele razy. To jedna z najważniejszych technik porządkowania programu. Zamiast kopiować ten sam zestaw instrukcji w kilku miejscach, definiujesz go raz i potem tylko wywołujesz.

Na poziomie praktycznym funkcje w MoonChunk przydają się m.in. do:

- budowania podpisów i etykiet,
- obliczeń pomocniczych,
- przygotowywania wartości przed wstawieniem do HTML,
- enkapsulacji logiki warunkowej,
- reuzycia tych samych transformacji w wielu stronach.

13.2 Klasyczna deklaracja funkcji

Podstawowa składnia wygląda tak:

```
1 function greet(name: string): string {  
2     return "Czesc, " + name;  
3 }
```

Listing 13.1: Klasyczna funkcja w MoonChunk

Mamy tu kilka elementów:

- słowo `function`,
- nazwe funkcji, tutaj `greet`,
- liste parametrow w nawiasach,
- opcjonalny typ zwracany po dwukropku,
- ciało funkcji w klamrach.

13.3 Parametry funkcji

Parametry to dane przekazywane do funkcji przy wywołaniu.

```
1 function fullTitle(title: string, suffix: string): string {  
2     return title + " - " + suffix;  
3 }
```

Listing 13.2: Funkcja z dwoma parametrami

Wywołanie może wyglądać tak:

```
1 let result: string = fullTitle("MoonChunk", "Docs");
```

Listing 13.3: Wywołanie funkcji

Funkcja dostaje argumenty przez wartość. Dla użytkownika języka oznacza to przede wszystkim tyle, że przy każdym wywołaniu pracuje na danych przekazanych do tego konkretnego wywołania.

13.4 Instrukcja `return`

`return` kończy działanie funkcji i oddaje wynik na zewnątrz.

```
1 function square(n: int): int {  
2     return n * n;  
3 }
```

Listing 13.4: Zwracanie wyniku z funkcji

Jeśli funkcja ma coś obliczyć, `return` jest jej naturalnym finałem.

13.5 Funkcje bez zwracanej wartosci

Nie kazda funkcja musi cos oddawac. Czasem celem jest tylko efekt uboczny, np. wypisanie czegos do konsoli przez `print`. Wtedy przydaje sie typ `void`.

```
1 function logMessage(text: string): void {  
2     print(text);  
3     return;  
4 }
```

Listing 13.5: Funkcja typu void

13.6 Funkcje a HTML

Najczestszy praktyczny wzorzec polega na tym, ze funkcja przygotowuje wartosc, a `content` tylko ja wyswietla.

```
1 function badge(isPublished: bool): string {  
2     return isPublished ? "Opublikowany" : "Szkic";  
3 }  
4  
5 page "" {  
6     let status: bool = true;  
7     content {  
8         <p>Status: {badge(status)}</p>  
9     };  
10 };
```

Listing 13.6: Funkcja przygotowujaca tekst do strony

To bardzo czytelne rozwiazanie. Logika jest zamknieta w funkcji, a HTML pozostaje prosty.

13.7 Funkcje jako granica zasiegu

Kazde wywołanie funkcji tworzy osobna przestrzen dla jej lokalnych nazw. Dlatego ten sam identyfikator moze bezpiecznie istniec i na zewnatrz, i wewnatrz funkcji.

```
1 const x: int = 3;  
2  
3 function demo(): int {  
4     const x: int = 5;
```

```
5   return x;  
6 }
```

Listing 13.7: Ta sama nazwa poza i wewnątrz funkcji

To bardzo przydatne, bo pozwala tworzyć funkcje samodzielne i niekolidujące z otoczeniem.

13.8 Rekurencja

Rekurencja oznacza, że funkcja wywołuje sama siebie. Brzmi tajemniczo, ale idea jest prosta: problem rozwiązuje się poprzez coraz mniejszą wersję tego samego problemu.

Klasyczny przykład:

```
1 function factorial(n: int): int {  
2     if (n == 0) {  
3         return 1;  
4     };  
5     return n * factorial(n - 1);  
6 }
```

Listing 13.8: Rekurencyjna funkcja factorial

Tutaj:

- gdy `n == 0`, funkcja zna odpowiedź od razu,
- w przeciwnym razie sprowadza problem do `factorial(n - 1)`.

13.9 Warunek stopu w rekurencji

To absolutnie kluczowe. Każda funkcja rekurencyjna musi mieć **warunek stopu**, który powoduje, że w pewnym momencie przestaje wywoływać sama siebie.

Bez tego program wpadłby w nieskończoną sekwencję wywołań.

13.10 Rekurencja do przechodzenia po kolejnych krokach

MoonChunk dobrze radzi sobie z funkcjami, które wywołują same siebie i przy okazji wypisują stan kolejnych kroków.

```
1 function walk(index: int, max: int): int {  
2     print("recursive index:", index);  
3     return (index >= max) ? index : walk(index + 1, max);  
4 }
```

Listing 13.9: Rekurencja z licznikiem lokalnym

To dobry przykład z dwóch powodów:

- pokazuje warunek stopu,
- pokazuje, że każde wywołanie ma swój lokalny zestaw argumentów i zmiennych.

13.11 Zapis strzałkowy

MoonChunk wspiera też skróconą, bardziej zwięzłą składnię funkcji.

```
1 sum(a: int, b: int): int => a + b;
```

Listing 13.10: Funkcja w składni strzałkowej

Mozesz też zwrócić blok instrukcji:

```
1 formatTitle(title: string): string => {  
2     return "[DOCS] " + title;  
3 };
```

Listing 13.11: Strzałka z blokiem instrukcji

W praktyce składnia strzałkowa jest dobra dla krótkich funkcji pomocniczych. Jeśli logika rośnie, klasyczne `function` bywa czytelniejsze.

13.12 Funkcje jako wartości

MoonChunk pozwala traktować funkcje jak wartości, co oznacza, że można je np. wypisywać przez `print`. To przydatne diagnostycznie i edukacyjnie.

```
1 function greet(name: string): string {  
2     return "Hello, " + name;  
3 }  
4  
5 print(greet);
```

```
6 print(greet("MoonChunk"));
```

Listing 13.12: Wypisanie obiektu funkcji i wyniku jej wywołania

Pierwsza linia pokazuje funkcję jako obiekt/byt wykonywalny, druga – wynik działania.

13.13 Najczęstsze błędy przy funkcjach

- brak `return` tam, gdzie funkcja ma oddać wynik,
- zwracanie wartości niezgodnej z zadeklarowanym typem,
- wywołanie czegoś, co nie jest funkcją,
- redefinicja nazw parametrów albo lokalnych zmiennych,
- brak warunku stopu w rekurencji.

13.14 Kiedy wydzielić funkcję

Jeśli ten sam fragment logiki pojawia się w dwóch miejscach albo jeśli blok `content` zaczyna być zanieczyszczony obliczeniami, to bardzo dobry moment, żeby wydzielić funkcję. Dobrze nazwana funkcja poprawia czytelność bardziej, niż na pierwszy rzut oka mogłoby się wydawać.

Rozdział 14

Importy, @include i składanie projektu z wielu plików

14.1 Dlaczego w ogóle dzielic projekt na pliki

Na początku jeden plik wydaje się wygodny. Jednak wraz z rozwojem projektu zaczyna być za duży: miesza metadata, logikę, strony, dane pomocnicze i drobne narzędzia. Wtedy naturalnym krokiem jest podział na mniejsze elementy.

MoonChunk obsługuje taki podział przez:

- `import`,
- `export chunk`,
- `@include`,
- jawne wywołania `moon(...)`.

14.2 Importy muszą stać na samej górze pliku

To jedna z zasad organizacyjnych MoonChunk. Importy powinny pojawić się przed deklaracjami chunków. Dzięki temu od razu widać zależności danego pliku.

```
1 import { Shared } from "../shared.mncnk";
2 import * as Pages from "../pages.mncnk";
3
4 chunk "Main" {
5   output: "../dist";
6 };
```

Listing 14.1: Importy na samej gorze pliku

To nie jest tylko kwestia estetyki. Taki układ wspiera przewidywalne parsowanie i czytelna organizację kodu.

14.3 Eksportowany chunk

Aby chunk mógł być importowany z innego pliku, powinien być oznaczony słowem `export`.

```
1 export chunk "Metadata" {  
2   title: "MoonChunk";  
3 };
```

Listing 14.2: Chunk eksportowany do innych plików

Taki zapis komunikuje jasno: ten chunk jest elementem publicznym, z którego inne pliki mogą korzystać.

14.4 Import nazwany

Import nazwany pozwala pobrać konkretne chunki po nazwie.

```
1 import { Metadata } from "./metadata.mncnk";
```

Listing 14.3: Import nazwany

To dobry wybór wtedy, gdy potrzebujesz jednego lub kilku konkretnych elementów.

14.5 Import przestrzeni nazw

Druga forma to import całej przestrzeni nazw pod aliasem.

```
1 import * as Fragments from "./fragments.mncnk";
```

Listing 14.4: Import przestrzeni nazw

Potem możesz odwoływać się do elementów przez kropkę, np. `Fragments.PageHeader`. To szczególnie wygodne, gdy z jednego pliku chcesz udostępnić większą kolekcję chunków.

14.6 Samo import niczego jeszcze nie uruchamia

To bardzo ważna zasada: `import` jedynie sprawia, że chunki stają się dostępne. Nie oznacza automatycznego wykonania.

Jezeli zaimportujesz plik, ale nie użyjesz żadnego z jego chunków przez `@include` albo `moon(...)`, to ten kod nie zostanie uruchomiony tylko dlatego, że istnieje. To zachowanie celowo chroni projekt przed ukrytymi efektami ubocznymi.

14.7 Do czego służy @include

`@include` włącza zawartość jednego chunku do drugiego. To bardzo wygodne przy składaniu wspólnych metadanych, wspólnej logiki lub fragmentów konfiguracyjnych.

```

1  import { Metadata } from "../metadata.mncnk";
2
3  chunk "Main" {
4    @include Metadata;
5    output: "../dist";
6  };

```

Listing 14.5: Dolaczenie wspólnego chunku

Możesz o tym myśleć jak o świadomym powiedzeniu: *do tego chunku dołącz też ten inny, wspólny zestaw instrukcji*.

14.8 @include powinno pojawiać się na początku chunku

MoonChunk narzuca uporządkowany styl: instrukcje `@include` powinny stać na samym początku ciała chunku. To sprawia, że od razu widać, od czego dany chunk zależy.

Taki układ jest czytelny i wart zachowania nawet wtedy, gdy projekt rośnie.

14.9 Import nazwany a import przez namespace

Różnica praktyczna jest prosta.

Import nazwany

```
1 import { Metadata } from "../metadata.mncnk";
2
3 chunk "Main" {
4     @include Metadata;
5 };
```

Listing 14.6: Użycie importu nazwanego

Import przestrzeni nazw

```
1 import * as Fragments from "../fragments.mncnk";
2
3 chunk "Main" {
4     @include Fragments.Metadata;
5 };
```

Listing 14.7: Użycie importu namespace

Pierwsza forma jest krótsza. Druga jest bardziej jawna i wygodna, gdy importujesz więcej rzeczy z jednego miejsca.

14.10 Lancuch importów

MoonChunk obsługuje projekty, w których jeden plik importuje drugi, a ten kolejny. Warto jednak zachować porządek i unikać zbędnej głębokości. Im prostszy graf zależności, tym łatwiej zrozumieć projekt.

14.11 Cykliczne zależności

Jeśli dwa pliki zaczęłyby importować się wzajemnie w niebezpieczny sposób, MoonChunk powinien zgłosić błąd związany z pętlą importów. To dobra ochrona przed konfiguracją, która byłaby bardzo trudna do przewidzenia dla użytkownika.

14.12 Punkt wejścia nadal musi być jawny

Nawet jeśli projekt składa się z wielu plików i wielu chunków, nadal ostatecznie ktoś musi wybrać, co ma zostać wykonane. Robi to `moon(...)`.


```
1 moon(Main);  
2 moon(Secondary);
```

Listing 14.8: Jawne wywołania punktów wejścia

Dzięki temu importy i `@include` odpowiadają za *dostępność i kompozycję*, a `moon(...)` odpowiada za *faktyczne uruchomienie*.

14.13 Typowy wzorzec modularnego projektu

Bardzo praktyczny układ wygląda tak:

```
1 content/  
2   site.mncnk  
3   metadata.mncnk  
4   sections.mncnk  
5   blog.mncnk  
6   data/  
7     posts.json
```

Listing 14.9: Przykładowy projekt wieloplikowy

Możliwe role:

- `site.mncnk` – główny plik z punktami `moon(...)`,
- `metadata.mncnk` – pola wspólne dla całej witryny,
- `sections.mncnk` – dodatkowe chunki do dołączania,
- `blog.mncnk` – logika stron blogowych,
- `posts.json` – dane wczytywane przez `data(...)`.

14.14 Najczęstsze błędy przy importach

- import w środku pliku zamiast na gorze,
- probowanie importu chunku bez `export`,
- literówka w nazwie importowanego chunku,
- probowanie użycia aliasu namespace, który nie istnieje,
- oczekiwanie, że sam import uruchomi generowanie strony.

14.15 Jak myśleć o @include

Najprostsza intuicja jest taka: `import` otwiera dostęp do chunku, a `@include` mówi, żeby ten chunk realnie dołączyć do aktualnego miejsca. To rozdzielanie jest bardzo zdrowe architektonicznie, bo oddziela *zależność* od *wykonania*.

Rozdział 15

Tablice, obiekty, dane z JSON i model rekordow

15.1 Dlaczego dane złożone są potrzebne

Prosta strona może opierać się na kilku pojedynczych zmiennych. Jednak prawdziwe projekty pracują zwykle na bogatszych strukturach: listach wpisów, produktach, autorach, kartach, menu czy konfiguracjach. Do takich rzeczy MoonChunk daje dwa podstawowe narzędzia:

- `array` dla uporządkowanych kolekcji,
- `object` oraz `dict` dla danych nazwanych.

15.2 Tablice

Tablica przechowuje elementy pod indeksami. Przykład:

```
1  const colors: array = ["red", "green", "blue"];
2  print(colors[0]);
```

Listing 15.1: Prosta tablica

To naturalny wybór wtedy, gdy porządek elementów ma znaczenie albo gdy chcesz iterować po danych pętlą z licznikiem.

15.3 Czy tablica musi być jednorodna

MoonChunk jest tutaj dość elastyczny. Tablica może zawierać wartości różnych typów, choć w praktyce dla czytelności zwykle lepiej trzymać w niej elementy podobnego rodzaju.

```
1 const mixed: array = [1, "ala", true];
```

Listing 15.2: Tablica mieszana

Technicznie jest to możliwe, ale projektowo warto zachować umiar.

15.4 Obiekty i słowniki

Obiekt przechowuje pola po nazwie.

```
1 const author: object = {  
2   name: "Alicja",  
3   role: "writer"  
4 };
```

Listing 15.3: Prosty obiekt

Słownik typu `dict` działa podobnie i jest szczególnie wygodny, gdy chcesz wyraźnie zaakcentować, że pracujesz na strukturze danych klucz–wartość.

```
1 let point: dict = { x: 1, y: 2 };
```

Listing 15.4: Słownik `dict`

15.5 Dostęp do pól przez kropkę

Najwygodniejszy i najczytelniejszy sposób pracy z obiektem to składnia kropkowa.

```
1 print(author.name);  
2 print(point.x);
```

Listing 15.5: Dostęp do pola obiektu

To dobry domyślny styl, gdy nazwa pola jest znana na etapie pisania kodu.

15.6 Dostęp przez nawiasy kwadratowe

Czasem klucz jest dynamiczny, np. przechowywany w innej zmiennej. Wtedy przydaje się składnia indeksowana.

```
1 let key: string = "name";
2 print(author[key]);
```

Listing 15.6: Dynamiczny dostęp do pola

Taki zapis jest bardzo mocny, bo pozwala programowi decydować, po które pole ma sięgnąć.

15.7 Klucze numeryczne i nietypowe

MoonChunk pozwala tworzyć obiekty i słowniki także z kluczami liczbowymi albo tekstowymi, które nie są zwykłymi identyfikatorami.

```
1 const details = {
2   stats: { total: 3, labels: ["alpha", "bravo", "charlie"] },
3   "meta-key": "enabled",
4   101: "numeric-key"
5 };
```

Listing 15.7: Nietypowe klucze obiektu

To przydaje się przy pracy z danymi przychodzącymi z zewnątrz, gdzie klucze nie zawsze są idealnie dopasowane do składni zwykłych identyfikatorów.

15.8 Modyfikowanie pól i elementów

MoonChunk pozwala nie tylko odczytywać dane z obiektów i tablic, ale też je zmieniać.

```
1 details.stats.total = details.stats.total + 1;
2 details.stats.labels[1] = "beta";
```

Listing 15.8: Modyfikacja danych złożonych

To bardzo wygodne, gdy chcesz przetworzyć dane przed wygenerowaniem finalnego HTML-a.

15.9 Tablice i obiekty jako model rekordow

MoonChunk nie wprowadza osobnej skladni typu *krotka* albo *rekord*. W praktyce rolę rekordu z nazwanymi polami pelni obiekt albo `dict`.

Przyklad punktu jako rekordu:

```
1  const point: object = { x: 1, y: 2 };
```

Listing 15.9: Rekord modelowany przez obiekt

Jesli potrzebujesz nazwanych pol, to zwykle jest wlasnie najlepsze rozwiazanie. Jesli potrzebujesz uporządkowanej listy bez nazw pol, uzyj tablicy.

15.10 Wczytywanie danych z pliku JSON przez `data(...)`

To jedna z najpraktyczniejszych funkcji w MoonChunk. Pozwala pobrac dane z pliku JSON i pracowac na nich jak na zwyklych strukturach danych.

```
1  const posts = data("./data/posts.json");
```

Listing 15.10: Wczytanie JSON do zmiennej

Po takim zapisie `posts` staje sie wartoscia, z ktora mozna pracowac w petlach, warunkach i interpolacjach.

15.11 Przykładowy plik JSON

```
1  [  
2    {  
3      "title": "Pierwszy wpis",  
4      "slug": "pierwszy-wpis",  
5      "published": true,  
6      "views": 120  
7    },  
8    {  
9      "title": "Drugi wpis",  
10     "slug": "drugi-wpis",  
11     "published": false,  
12     "views": 30  
13   }  
14 ]
```

Listing 15.11: Przykładowe dane posts.json

Taki plik mozesz potem przeglądać w MoonChunk przez indeks i pola obiektu.

15.12 Przykład pracy z danymi JSON

```
1  const posts = data("./data/posts.json");
2
3  page "" {
4    for (let i: int = 0; i < 2; i++) {
5      if (posts[i].published) {
6        content {
7          <article>
8            <h2>{posts[i].title}</h2>
9            <p>Views: {posts[i].views}</p>
10           </article>
11         };
12       };
13     };
14   };
```

Listing 15.12: Filtrowanie i wyswietlanie wpisow

To prosty, ale bardzo realny scenariusz: część danych jest pokazywana tylko wtedy, gdy dany wpis jest opublikowany.

15.13 Kiedy uzywac object, a kiedy dict

Z perspektywy czystego uzytkowania oba typy sa do siebie bardzo podobne. Warto przyjac prostą konwencje:

- `object` – gdy myslisz o konkretnej strukturze modelu,
- `dict` – gdy myslisz o slowniku danych i kluczach.

Nie jest to sztywna roznica filozoficzna, ale bardzo pomaga w czytelności projektu.

15.14 Typowe błędy przy danych złożonych

- próbowanie odczytu pola z `null` albo `undefined`,
- niepoprawny typ indeksu w nawiasach kwadratowych,
- zakładanie, że dana struktura ma pole, którego nie ma,
- próbowanie traktowania prostej liczby jak obiektu,
- wpisywanie skomplikowanej logiki danych bezpośrednio do HTML-a zamiast przygotować wartości wcześniej.

15.15 Dobra praktyka: dane najpierw, HTML potem

Przy pracy z tablicami i obiektami bardzo opłaca się najpierw przygotować czyste dane, a dopiero potem budować z nich treść strony. To redukuje bałagan i sprawia, że `content` pozostaje głównie warstwą prezentacji, a nie mieszaniną logiki i znaczników.

Rozdział 16

Print, diagnostyka i czytanie błędów

16.1 Po co istnieje `print(...)`

Gdy uczysz się nowego języka albo rozwijasz bardziej złożony projekt, bardzo często chcesz po prostu zobaczyć, co dzieje się w środku programu. Właśnie po to istnieje funkcja wbudowana `print(...)`.

Działa ona jak narzędzie podglądu. Pozwala wypisać wartości do konsoli podczas wykonania MoonChunk.

16.2 Najprostsze użycie

```
1 print("Hello");  
2 print(1, 1.0, true, "text");
```

Listing 16.1: Podstawowe wypisywanie wartości

Mozesz przekazać jeden argument albo kilka argumentów jednocześnie.

16.3 Wypisywanie pustej linii

`print()` bez argumentów też ma sens. Pozwala odseparować grupy logów i zrobić czytelniejszy output w terminalu.

```
1 print("Sekcja A");  
2 print();  
3 print("Sekcja B");
```

Listing 16.2: Pusta linia przez `print()`

16.4 Co można wypisać

MoonChunk stara się być tutaj elastyczny. Możesz wypisywać:

- liczby,
- tekst,
- wartości logiczne,
- tablice,
- obiekty,
- wynik wywołania funkcji,
- sama funkcje,
- nawet chunki jako obiekty logiczne.

```
1 function greet(name: string): string {  
2     return "Hello, " + name;  
3 }  
4  
5 print(greet);  
6 print(greet("MoonChunk"));  
7 print(Main);
```

Listing 16.3: Różne rodzaje argumentów print

To przydatne szczególnie przy debugowaniu i poznawaniu modelu wykonania języka.

16.5 Print jako narzędzie edukacyjne

Na początku nauki `print(...)` jest jednym z najlepszych sposobów zrozumienia programu. Dzięki niemu możesz sprawdzać:

- czy zmienna ma taką wartość, jakiej oczekujesz,
- czy warunek daje `true` czy `false`,
- jak zmienia się licznik w pętli,
- czy funkcja zwraca poprawny wynik,
- jakie dane zostały wczytane z pliku JSON.

16.6 Jak wygląda dobry log

Dobre logowanie nie polega na wypisywaniu wszystkiego bez planu. Lepiej użyć krótszych, konkretnych komunikatów.

```
1 print("counter:", counter);  
2 print("current post slug:", posts[i].slug);  
3 print("isPublished:", posts[i].published);
```

Listing 16.4: Czytelny styl logowania

Taki zapis pozwala od razu zrozumieć, na co patrzysz.

16.7 Błędy składniowe a błędy wykonania

MoonChunk może zgłaszać błędy na różnych etapach. Z perspektywy użytkownika warto rozróżniać dwie duże kategorie.

Błędy składniowe

To problemy takie jak:

- brak średnika,
- niezamknięty nawias,
- nieoczekiwany token,
- literówka w konstrukcji języka.

Błędy wykonania

To problemy, które pojawiają się dopiero podczas realnego wykonywania kodu, np.:

- nieznaną zmienną,
- niezgodność typu,
- dzielenie przez zero,
- odczyt z `null`,

- użycie `break` poza pętlą,
- brak inicjalizacji `let` przed odczytem.

16.8 Jak czytać komunikat diagnostyczny

Dobry komunikat MoonChunk zwykle zawiera:

- informacje, co poszło nie tak,
- numer linii,
- numer kolumny,
- czasem sugestie dotycząca literówki.

Nie czytaj takiego komunikatu jak kary. Traktuj go jak wskazówkę. Najczęściej da się z niego wyciągnąć bardzo konkretną informację o miejscu i naturze problemu.

16.9 Typowe komunikaty i ich znaczenie

Nieznana zmienna

Jeśli MoonChunk mówi, że zmienna nie jest zadeklarowana, najczęściej oznacza to jedno z trzech:

- literówkę w nazwie,
- próbę użycia zmiennej poza zasięgiem,
- próbę użycia zmiennej zanim została utworzona.

Zmienna zadeklarowana, ale nie zainicjalizowana

Ten komunikat pojawia się, gdy `let` został utworzony bez wartości, a potem odczytany za wcześnie.

Niezgodność typu

Oznacza, że interpreter spodziewał się jednego rodzaju wartości, a dostał inny. Najczęściej trzeba poprawić deklarację typu albo wykonać świadome rzutowanie.

Błąd sterowania przepływem

Dotyczy np. `break` albo `continue` poza pętlą.

16.10 Strategia debugowania krok po kroku

Gdy coś nie działa, warto przyjąć prosty plan:

1. znajdź pierwszą linię, na której pojawia się błąd,
2. sprawdź, jakie wartości biorą w nim udział,
3. dodaj `print(...)` przed problematycznym miejscem,
4. upewnij się, że typy są takie, jak myślisz,
5. uprość fragment do mniejszego przykładu i sprawdź go osobno.

To podejście jest nudne, ale niezwykle skuteczne.

16.11 Jak odróżnić błąd w logice od błędu w danych

Czasem problem nie leży w samej składni programu, tylko w tym, co przychodzi z danych. Przykładowo:

- JSON może nie mieć pola, które zakładasz,
- liczba może przyjść jako tekst,
- element listy może być pusty,
- flaga logiczna może mieć nieoczekiwany stan.

W takich sytuacjach `print(...)` jest nieoceniony.

16.12 Dobra praktyka: loguj stan, nie emocje

Zamiast wypisywać rzeczy typu "ups" albo "cos nie działa", lepiej logować konkretne stany.

```
1 print("post index:", i);  
2 print("post title:", posts[i].title);  
3 print("published:", posts[i].published);
```

Listing 16.5: Przykład konkretnego logu

Taki log od razu pomaga znaleźć przyczynę problemu.

16.13 Kiedy usuwać logi

W projekcie edukacyjnym część logów warto zostawić na dłużej. W projekcie produkcyjnym lepiej usuwać albo ograniczać logi diagnostyczne, gdy przestają być potrzebne. Kod jest wtedy cichszy i łatwiejszy do czytania.

Rozdział 17

Dobre praktyki pracy z MoonChunk

17.1 Po co w ogóle mówić o praktykach

Sama znajomość składni nie wystarcza, aby pisać dobry kod. Można stworzyć program, który technicznie działa, ale jest trudny do czytania, zły do rozbudowy i nieprzyjemny w utrzymaniu. Dobre praktyki sprawiają, że projekt pozostaje czytelny po tygodniu, miesiącu i semestrze.

17.2 Oddzielaj dane od prezentacji

Jeden z najlepszych nawyków w MoonChunk polega na tym, aby najpierw przygotować dane, a dopiero potem generować z nich HTML.

Lepszy styl:

```
1 let heading: string = "Aktualnosci";
2 let intro: string = "Najnowsze wpisy z projektu";
3
4 content {
5   <h1>{heading}</h1>
6   <p>{intro}</p>
7 };
```

Listing 17.1: Najpierw dane, potem content

Gorszy styl to upychanie wszystkiego bezpośrednio do jednego, ogromnego bloku `content`.

17.3 Nazywaj rzeczy tak, jak je rozumiesz

Nazwy typu `x`, `tmp`, `data2`, `something` szybko zamieniają projekt w labirynt. Dobra nazwa bardzo pomaga, nawet jeśli jej wpisanie zajmie trzy sekundy dłużej.

Dobre przykłady:

- `siteTitle`,
- `isPublished`,
- `cardsHtml`,
- `featuredPosts`,
- `buildAuthorLabel`.

17.4 Trzymaj metadata razem

Jeśli kilka stron korzysta z tych samych pól metadata, wydziel osobny chunk z konfiguracją wspólną i dołącz go przez `@include`. To jeden z najłatwiejszych sposobów na uniknięcie kopiowania kodu.

17.5 Używaj funkcji do logiki, nie do wszystkiego

Funkcje są świetne, ale nie muszą zastąpić całej struktury projektu. Najlepiej sprawdzają się wtedy, gdy:

- masz powtarzalne obliczenie,
- chcesz nazwać fragment logiki,
- ten sam schemat pojawia się w wielu miejscach,
- chcesz odchudzić blok `content`.

Jeśli jednak funkcja zaczyna przejmować odpowiedzialność za pole projektu naraz, może to być znak, że lepiej wydzielić osobny chunk albo osobny plik.

17.6 Ograniczaj magię w wyrażeniach

Bardzo długie warunki i skomplikowane wyrażenia wstawione bezpośrednio do HTML-a wyglądają sprytnie tylko przez chwilę. Potem stają się meczące.

Zamiast tego:

```
1 let canShowBanner: bool = isPublished and not isArchived;  
2 let bannerLabel: string = canShowBanner ? "Widoczny" : "Ukryty";
```

Listing 17.2: Lepsza czytelność przez nazwy pomocnicze

17.7 Utrzymuj pliki małe i sensowne

Jeśli plik robi zbyt wiele rzeczy naraz, podziel go. Dobry podział nie wynika z sztywnej liczby linii, ale z odpowiedzialności. Przykładowo osobny plik może mieć:

- wspólne metadata,
- logikę bloga,
- zestaw chunków sekcyjnych,
- dane zewnętrzne.

17.8 Używaj `print(...)` świadomie

Debugowanie jest normalną częścią pracy, ale logi też potrafią zasmiecić projekt. Zostawiaj te, które rzeczywiście pomagają, i usuwaj te jednorazowe, gdy przestają być potrzebne.

17.9 Nie nadużywaj globali

Globalne wartości są wygodne, ale jak każde wygodne narzędzie mogą być nadużywane. Nie wszystko musi być globalne. Trzymaj globalnie tylko to, co naprawdę wspólne dla wielu miejsc programu.

17.10 Wybieraj prostszy typ, jeśli wystarczy

Nie trzeba wszędzie wpisywać `unknown`, `any` czy rozbudowanych rzutowań. Jeśli wystarczy `string`, `int`, `array` albo `object`, wybierz właśnie je. Kod będzie prostszy i bezpieczniejszy.

17.11 Zanim rozbudujesz, zbuduj mały pionowy przekroj

Praktyczna rada projektowa: zanim napiszesz całą architekturę, zbuduj mały kawałek, który przechodzi przez wszystkie warstwy. Na przykład:

1. jedna strona,
2. jeden chunk wspólny,
3. jedno źródło JSON,
4. jedna pętla,
5. jeden warunek,
6. jedna funkcja pomocnicza.

Taki przekroj daje pewność, że fundament działa.

17.12 Komentuj tam, gdzie jest decyzja, nie tam, gdzie jest oczywistość

Jeśli dodajesz komentarz, niech wyjaśnia *dla czego*, a nie tylko *co*.

Dobry komentarz:

```
1 // Pokazujemy tylko wpisy opublikowane, bo szkice nie powinny trafiać do HTML-  
  a.  
2 if (posts[i].published) {  
3     ...  
4 };
```

Listing 17.3: Komentarz wyjaśniający decyzje

Słabszy komentarz to taki, który tylko powtarza kod.

17.13 Buduj projekt tak, jakby miał urosnąć

Nawet jeśli dziś projekt ma dwie podstrony, jutro może mieć dwadzieścia. Nie chodzi o przedwczesne komplikowanie, tylko o zdrowe podstawy:

- sensowny podział plików,
- dobre nazwy,
- wspólne metadane,
- funkcje dla powtarzalnej logiki,
- dane ładowane z zewnątrz tam, gdzie to ma sens.

To zwykle wystarczy, aby projekt rozwijał się spokojnie i bez chaosu.

Rozdział 18

Projekt od zera: budujemy maly serwis krok po kroku

18.1 Cel projektu

Na koniec przejdziemy przez cały, realistyczny mini-projekt. Zbudujemy prosty serwis złożony z:

- strony glownej,
- listy wpisow,
- wspolnych metadata,
- funkcji pomocniczych,
- danych w pliku JSON,
- kilku importowanych chunkow.

Naszym celem nie jest stworzenie ogromnej aplikacji, tylko pokazanie, jak elementy Moon-Chunk lacza sie w sensowna calosc.

18.2 Struktura plikow projektu

Przyjmijmy taki ukklad:

```
1 book-demo/  
2   content/  
3   site.mncnk
```

```
4  metadata.mncnk
5  blog.mncnk
6  sections.mncnk
7  data/
8    posts.json
9  run.js
```

Listing 18.1: Struktura przykładowego projektu

18.3 Dane wpisów

Zacznijmy od danych. Plik `posts.json` może wyglądać tak:

```
1  [
2    {
3      "title": "MoonChunk od podstaw",
4      "slug": "moonchunk-od-podstaw",
5      "description": "Pierwszy kontakt z językiem i podstawowa składnia.",
6      "published": true,
7      "views": 120
8    },
9    {
10     "title": "Funkcje i petle",
11     "slug": "funkcje-i-petle",
12     "description": "Jak przetwarzać wiele elementów i porządkować logikę.",
13     "published": true,
14     "views": 90
15   },
16   {
17     "title": "Wpis roboczy",
18     "slug": "wpis-roboczy",
19     "description": "Jeszcze nie powinien być pokazany publicznie.",
20     "published": false,
21     "views": 5
22   }
23 ]
```

Listing 18.2: Przykładowy plik `posts.json`

18.4 Wspólne metadata

Następnie wydzielmy wspólną konfigurację strony do osobnego chunku.

```
1 export chunk "Metadata" {
2   lang: "pl";
3   charset: "utf-8";
4   viewport: "width=device-width, initial-scale=1";
5   metaAuthor: "MoonChunk Team";
6   metaRobots: "index,follow";
7   themeColor: "#111827";
8   styles: "<link rel='stylesheet' href='/assets/app.css'>";
9   footer: "<footer><small>MoonChunk Demo</small></footer>";
10 };
```

Listing 18.3: Plik metadata.mnck

To bardzo praktyczny krok. Dzięki temu nie musimy powtarzać tych samych pól w wielu chunkach.

18.5 Wspólna sekcja powitalna

Zrobmy też dodatkowy chunk z sekcją powitalną.

```
1 export chunk "HeroSection" {
2   page "" {
3     content {
4       <section class="hero">
5         <h1>MoonChunk Blog</h1>
6         <p>Statyczny serwis budowany z danych i logiki.</p>
7       </section>
8     };
9   };
10 };
```

Listing 18.4: Plik sections.mnck

W tym miejscu warto zwrócić uwagę na ciekawy aspekt: chunk może zawierać w sobie `page`. Oznacza to, że logika strony może być składana z mniejszych części.

18.6 Logika bloga

Teraz utworzymy osobny plik odpowiedzialny za generowanie strony z wpisami.

```
1  export chunk "BlogPages" {
2    const posts = data("./data/posts.json");
3
4    function badge(views: int): string {
5      return views > 100 ? "Popularny" : "Nowy";
6    }
7
8    page "blog" {
9      content {
10        <section>
11          <h2>Lista wpisow</h2>
12        </section>
13      };
14
15      for (let i: int = 0; i < 3; i++) {
16        if (posts[i].published) {
17          content {
18            <article class="post-card">
19              <h3>{posts[i].title}</h3>
20              <p>{posts[i].description}</p>
21              <p>Status: {badge(posts[i].views)}</p>
22              <p>Views: {posts[i].views}</p>
23              <a href={"/posts/" + posts[i].slug}>Czytaj wiecej</a>
24            </article>
25          };
26        };
27      };
28    };
29  };
```

Listing 18.5: Plik blog.mnck

W tym fragmencie mamy od razu kilka istotnych elementów:

- wczytanie danych przez `data(...)`,
- funkcje pomocnicza,
- petle `for`,

- warunek if,
- dynamiczne atrybuty HTML,
- generowanie wielu podobnych kart.

18.7 Plik główny

Na koniec składamy wszystko w punkcie wejścia.

```

1  import { Metadata } from "../metadata.mnck";
2  import { BlogPages } from "../blog.mnck";
3  import * as Sections from "../sections.mnck";
4
5  chunk "Main" {
6    @include Metadata;
7    @include BlogPages;
8    output: "../dist";
9    title: "MoonChunk Blog";
10   metaDescription: "Przykładowy serwis statyczny zbudowany w MoonChunk.";
11
12   page "" {
13     content {
14       <section>
15         <h1>Start</h1>
16         <p>To jest strona główna serwisu.</p>
17       </section>
18     };
19   };
20 };
21
22 moon(Main);

```

Listing 18.6: Plik site.mnck

18.8 Co robi ten projekt

Po uruchomieniu projekt wygeneruje co najmniej:

- strone główna `index.html`,
- strone bloga `blog.html`,

- wspólny dokument z odpowiednimi metadata,
- HTML z kartami dla wpisów, które mają `published = true`.

Dzięki temu powstaje mały, ale realistyczny serwis statyczny.

18.9 Minimalny runner do tego projektu

```
1  const { executeMoonChunkFile } = require("moonchunk");
2
3  const result = executeMoonChunkFile("./content/site.mncnk");
4
5  if (!result.ok) {
6    console.error(result.diagnostics);
7    process.exit(1);
8  }
9
10 console.log(result.generatedFiles || []);
```

Listing 18.7: Plik `run.js` dla projektu

To wszystko. Od strony użytkownika tyle wystarczy, aby uruchomić generator.

18.10 Jak rozwijać ten projekt dalej

To dopiero początek. Z tego miejsca można iść w wielu kierunkach:

- dodać osobne strony `posts/...`,
- rozbudować metadata dla Open Graph i Twittera,
- dodać więcej funkcji pomocniczych,
- wprowadzić wspólne chunki dla sekcji strony,
- podzielić dane na kilka plików JSON,
- wzbogacić layout przez CSS i skrypty dołączane w metadata.

18.11 Czego ten projekt uczy

Ten projekt jest wartosciowy nie dlatego, że jest wielki, tylko dlatego, że przechodzi przez niemal wszystkie fundamenty MoonChunk:

- struktury plików,
- importy,
- eksportowane chunki,
- `@include`,
- `moon(...)`,
- metadane,
- funkcje,
- pętle i warunki,
- dane JSON,
- generowanie HTML.

Jeśli rozumiesz ten przykład od początku do końca, masz bardzo solidne podstawy do budowania własnych projektów w MoonChunk.

18.12 Ostatnia rada praktyczna

Kiedy tworzysz pierwszy prawdziwy projekt, nie próbuj od razu zbudować wszystkiego. Najpierw uruchom jedną stronę, potem dodaj wspólne metadane, potem dane, potem pętle, a dopiero na końcu bardziej zaawansowane elementy. To najspokojniejsza droga do opanowania języka.

Rozdział 19

Wzorce uzycia i gotowe przepisy

19.1 Po co w ogole wzorce

Gdy poznasz juz skladnie jezyka, kolejnym krokiem nie jest zapamietanie kolejnych tokenow, lecz nauczanie sie powtarzalnych sposobow rozwiazywania realnych problemow. Wzorzec to po prostu sprawdzony uklad kodu, ktory warto miec z tylu glowy.

19.2 Wzorzec: strona glowna z sekcjami

Bardzo czesty przypadek to jedna strona glowna z kilkoma sekcjami. W MoonChunk warto podzielic ja na przygotowanie danych i kilka blokow `content`.

```
1 chunk "Home" {
2   output: "./dist";
3   title: "Strona glowna";
4
5   let heroTitle: string = "MoonChunk";
6   let heroText: string = "Jezyk do generowania statycznych stron.";
7
8   page "" {
9     content {
10       <section class="hero">
11         <h1>{heroTitle}</h1>
12         <p>{heroText}</p>
13       </section>
14     };
15
16     content {
17       <section class="cta">
```

```
18     <a href="/docs">Przejdź do dokumentacji</a>
19   </section>
20   };
21 };
22 };
```

Listing 19.1: Strona główna składana z sekcji

Ten wzorzec jest prosty, czytelny i dobrze skaluje się do landing page'y.

19.3 Wzorzec: lista kart

Jeśli chcesz zbudować siatkę podobnych elementów, np. kart, pętla jest naturalnym wyborem.

```
1  const cards = [
2    { title: "Parser", description: "Rozpoznawanie składni." },
3    { title: "Runtime", description: "Wykonywanie programu." },
4    { title: "Output", description: "Generowanie HTML." }
5  ];
6
7  page "" {
8    for (let i: int = 0; i < 3; i++) {
9      content {
10         <article class="card">
11           <h3>{cards[i].title}</h3>
12           <p>{cards[i].description}</p>
13         </article>
14       };
15     };
16   };
```

Listing 19.2: Lista kart budowana z tablicy obiektów

Warto zauważyć, że najwygodniej przechowywać dane kart jako obiekty, a nie jako kilka osobnych tablic.

19.4 Wzorzec: stan pusty

To bardzo ważny scenariusz w praktyce. Jeśli lista jest pusta, strona nie powinna być po prostu biała i milcząca. Lepiej pokazać użytkownikowi jasny komunikat.

```
1  let hasItems: bool = false;
```

```
2
3  page "" {
4      if (hasItems) {
5          content {
6              <p>Mamy elementy do pokazania.</p>
7          };
8      } else {
9          content {
10             <p>Na razie brak elementow.</p>
11         };
12     };
13 };
```

Listing 19.3: Pusty stan listy

19.5 Wzorzec: filtr publikacji

W projektach treściowych bardzo często przechowujesz wpisy, które nie powinny jeszcze trafić do publicznego HTML-a. Najprostszy wzorzec wygląda tak:

```
1  for (let i: int = 0; i < 3; i++) {
2      if (posts[i].published) {
3          content {
4              <h2>{posts[i].title}</h2>
5          };
6      };
7  };
```

Listing 19.4: Pokazuj tylko elementy opublikowane

To nie tylko czytelne, ale i bezpieczne. Szkice nie wypływają przypadkiem do finalnej strony.

19.6 Wzorzec: funkcja do etykiety

Jeśli wiele miejsc potrzebuje podobnej etykiety, nie powtarzaj warunku w kółko.

```
1  function statusLabel(published: bool): string {
2      return published ? "Opublikowany" : "Szkic";
3  }
```

Listing 19.5: Funkcja etykietująca stan

Taki wzorzec bardzo dobrze działa dla statusów, podpisów i krótkich komunikatów.

19.7 Wzorzec: wspólne metadata plus lokalne nadpisanie

To jeden z najbardziej produktywnych wzorców w MoonChunk.

```
1  export chunk "Metadata" {
2    lang: "pl";
3    charset: "utf-8";
4    metaAuthor: "MoonChunk Team";
5  };
6
7  chunk "About" {
8    @include Metadata;
9    output: "../dist";
10   title: "0 projekcie";
11   metaDescription: "Opis strony o projekcie.";
12  };
```

Listing 19.6: Wspólne metadata i lokalne doprecyzowanie

Dzięki temu projekt ma centrum konfiguracji, ale pojedyncze strony nadal zachowują swoją tożsamość.

19.8 Wzorzec: oblicz najpierw, pokaz potem

To wzorzec, który warto stosować niemal wszędzie.

```
1  let canShowPromo: bool = views > 100 and isPublished;
2  let promoText: string = canShowPromo ? "Popularny wpis" : "Standardowy wpis";
3
4  content {
5    <p>{promoText}</p>
6  };
```

Listing 19.7: Oddzielenie logiki od prezentacji

Dzięki temu kod HTML nie jest przeciążony złożonymi wyrażeniami.

19.9 Wzorzec: rekurencyjny spacer po etapach

Rekurencja nie służy tylko do klasycznej silni. Można jej użyć też do przechodzenia po kolejnych krokach, np. numerowanych etapach procesu.

```
1 function walk(index: int, max: int): int {  
2     print("step:", index);  
3     return index >= max ? index : walk(index + 1, max);  
4 }
```

Listing 19.8: Rekurencja jako przechodzenie po krokach

To dobry wzorzec wszędzie tam, gdzie problem naturalnie rozkłada się na podobne kroki.

Rozdział 20

Antywzorce i pulapki

20.1 Po co poznawac antywzorce

Dobre praktyki mówią, co robić. Antywzorce pokazują, czego unikać. To równie ważne, bo wiele problemów nie wynika z braku wiedzy, tylko z pozornie wygodnych skrótów, które w dłuższej perspektywie psują czytelność albo stabilność projektu.

20.2 Antywzorzec: wszystko w jednym pliku

Początkujący często zaczynają od jednego ogromnego pliku i zostają przy nim zbyt długo. Skutek jest prosty: metadane, funkcje, dane, strony i debug printy mieszają się ze sobą.

Jeśli projekt zaczyna rosnąć, podziel go na chunki i pliki. Nie czekaj, aż zrobi się naprawdę nieczytelny.

20.3 Antywzorzec: kopiowanie metadane

Kopiowanie tych samych pól `title`, `metaDescription`, `lang` czy `styles` do wielu chunków prędzej czy później doprowadzi do rozjazdów. Jedna strona będzie miała stary opis, inna nowy, trzecia w ogóle bez aktualizacji.

Lepiej wydzielić wspólny chunk `Metadata` i używać `@include`.

20.4 Antywzorzec: zbyt duzo logiki w content

To jeden z najczestszych bledow. Jesli w srodku HTML-a masz kilka zagniezdzonych warunkow, dlugie rzutowania i budowanie tekstu na piec sposobow, to znak, ze logika powinna zostac przeniesiona wyzej do zmiennych albo funkcji.

20.5 Antywzorzec: naduzywanie unknown i any

Te typy sa potrzebne, ale nie powinny stac sie codziennym domyslnym rozwiazaniem. Jesli wszedzie uciekasz do `unknown` albo `any`, tak naprawde rezygnujesz z duzej części korzysci wynikajacych ze swiadomego modelu typow.

20.6 Antywzorzec: brak warunku stopu w rekurencji

To klasyczna pulapka. Rekurencja bez sensownego warunku stopu nie prowadzi do wyniku, tylko do spirali wywołan. Jesli piszesz funkcje wywołujaca sama siebie, pierwszym pytaniem powinno byc zawsze: *kiedy to sie zatrzyma?*

20.7 Antywzorzec: nieczytelne nazwy

Nazwy typu `tmp2`, `val`, `x1` i `something` nie pomagaja zrozumiec kodu. Zwlaszcza w DSL-u do generowania stron dobra nazwa potrafi zaoszczedzic bardzo duzo czasu.

20.8 Antywzorzec: traktowanie if jako miejsca do swobodnego cieniowania

MoonChunk celowo jest rygorystyczny w kwestii redeklaracji nazw w cialach `if`, `for` i `while`. Jesli sprobujesz tworzyc tam nowe zmienne o tych samych nazwach, które juz istnieja w poblizu, szybko trafisz na komunikat o redeklaracji.

Jesli naprawde potrzebujesz osobnego zakresu z ta sama nazwa, uzyj jawnego bloku `{ ... }` albo funkcji.

20.9 Antywzorzec: import bez realnego użycia

Samo importowanie pliku niczego nie uruchamia. Jeśli liczysz na to, że zaimportowany chunk wykona się sam, projekt zacznie wyglądać tak, jakby “nic nie działało”. Musisz go świadomie dołączyć przez `@include` albo uruchomić przez `moon(...)`.

20.10 Antywzorzec: brak strategii debugowania

Chaotyczne dodawanie losowych `print(...)` jest lepsze niż nic, ale nie dużo lepsze. Lepsza strategia to zadać sobie pytanie: jaką wartość chce sprawdzić i w którym miejscu? Dopiero wtedy dodaj log.

20.11 Antywzorzec: projekt budowany od końca

Częsta pomyłka polega na tym, że autor zaczyna od bardzo rozbudowanej struktury, zanim powstanie pierwszy działający przebieg. Lepiej zrobić odwrotnie: najpierw mały projekt, który naprawdę generuje HTML, a dopiero potem dokładać warstwy i modularyzację.

Rozdział 21

FAQ: najczęstsze pytania użytkowników

21.1 Czy jeden plik `.mncnk` może generować wiele stron

Tak. Wystarczy, że w odpowiednich chunkach zdefiniujesz więcej niż jedną instrukcję `page`, albo uruchomisz wiele punktów wejścia przez `moon(...)`.

21.2 Czy import od razu generuje HTML

Nie. Import tylko udostępnia chunki z innego pliku. Realne użycie następuje dopiero po `@include` albo `moon(...)`.

21.3 Czy muszę podawać `.html` w trasie strony

Nie. MoonChunk potrafi sam dopisać rozszerzenie `.html` tam, gdzie ma to sens. Dlatego zwykle zapisujesz po prostu `page "about"`.

21.4 Czy mogę pominąć wiodący ukośnik w trasie

Tak. MoonChunk normalizuje takie ścieżki. Dla użytkownika oznacza to, że zapis logiczny jest ważniejszy niż ręczne dbanie o format nazwy pliku wyjściowego.

21.5 Czy moge użyć `content` poza `page`

Nie w zwykłym sensie generowania strony. `content` jest przeznaczony do składania treści dokumentu strony. W praktyce pracujesz z nim w kontekście `page`.

21.6 Czy `let` bez wartości jest bezpieczne

Tak, o ile pamiętasz, że musisz nadać wartość przed odczytem. To narzędzie wygodne, ale wymaga dyscypliny.

21.7 Czy moge zmieniać `const`

Nie. Jeśli wiesz, że wartość ma się zmieniać, użyj `let`.

21.8 Czy moge mieć tę samą nazwę w wielu miejscach

Tak, ale nie wszędzie. Ta sama nazwa jest bezpieczna np. w osobnym chunku, funkcji albo jawnie wydzielonym bloku `{ ... }`. Nie należy zakładać, że takie samo cieniowanie będzie dozwolone w ciele `if`, `for` czy `while`.

21.9 Czy MoonChunk ma osobny typ rekordu albo krotki

Nie ma osobnej składni rekordu w sensie odrębnego typu językowego. W praktyce rolę rekordu pełni `object` albo `dict`. Jeśli potrzebujesz nazwanych pól, to zazwyczaj jest właśnie najlepszy wybór.

21.10 Czy dane z JSON są tylko do odczytu

MoonChunk pozwala pracować na danych i w wielu przypadkach je modyfikować w czasie wykonania programu, np. zmieniać pola obiektów i elementy tablic przed wygenerowaniem finalnego HTML-a.

21.11 Czy moze wypisac funkcje albo chunk przez `print`

Tak. `print(...)` potrafi wypisywac nie tylko podstawowe wartosci, ale tez bardziej zlozone byty, w tym funkcje i chunki.

21.12 Czy petle musza miec licznik typu `int`

W praktyce przy klasycznej petli `for` jest to najwygodniejsze i najbardziej naturalne. MoonChunk przewiduje właśnie taki czytelny, kontrolowany model iteracji.

21.13 Czy `break` i `continue` dzialaja wszedzie

Nie. Dzialaja tylko wewnatrz petli. Poza `for` i `while` sa traktowane jako blad.

21.14 Czy musze wszedzie podawac typy

Nie zawsze. W wielu przypadkach typ mozna pominac, jesli wynika on jasno z przypisanej wartosci. Jednak jawne typy bardzo pomagaja w czytelnosci i sa polecane w wazniejszych miejscach projektu.

21.15 Czy musze znac parser i ANTLR, zeby korzystac z MoonChunk

Nie. To wiedza potrzebna autorowi implementacji jezyka, a nie zwyklemu uzytkownikowi, ktory chce pisac strony i logike projektu.

Rozdział 22

Cwiczenia i propozycje samodzielnej pracy

22.1 Dlaczego warto robic cwiczenia

Sama lektura dokumentacji daje podstawy, ale prawdziwe zrozumienie przychodzi dopiero, gdy samodzielnie zmieniasz kod i obserwujesz efekt. Dlatego na koniec warto przejść przez kilka małych zadań.

22.2 Cwiczenie 1: prosta strona wizytowka

Cel: utwórz `chunk` generujący pojedynczą stronę główną z nagłówkiem, opisem i linkiem.

Spróbuj samodzielnie:

- dodać `title` i `metaDescription`,
- zdefiniować trzy zmienne tekstowe,
- wstawić je do `content`.

22.3 Cwiczenie 2: lista kart

Cel: zbuduj tablice trzech obiektów z polami `title`, `description`, `button` i wygeneruj z niej karty w petli `for`.

To ćwiczenie utrwała:

- tablice,
- obiekty,
- indeksowanie,
- dynamiczne `content` w petli.

22.4 Cwiczenie 3: filtr opublikowanych wpisow

Cel: przygotuj tablice wpisow z polem `published` i pokaz tylko te, ktore sa gotowe do publikacji. Dodaj tez tekst “brak wpisow”, jesli zadny nie przejdzie filtra.

22.5 Cwiczenie 4: funkcja etykietujaca

Cel: napisz funkcje, ktora na podstawie liczby wyswietlen zwraca etykiety, np. "Nowy", "Popularny", "Hit". Uzyj tej funkcji w HTML-u.

22.6 Cwiczenie 5: wczytanie JSON

Cel: przenies dane wpisow z kodu MoonChunk do pliku JSON i wczytaj je przez `data(...)`. Sprawdz przez `print(...)`, czy dane wygladaja tak, jak oczekujesz.

22.7 Cwiczenie 6: wspolne metadata

Cel: wydziel chunk `Metadata`, zaimportuj go do pliku glownego i nadpisz `title` dla jednej konkretnej strony.

22.8 Cwiczenie 7: rekurencja

Cel: napisz funkcje rekurencyjna, ktora wypisuje liczby od 0 do 5. Zadbaj o czytelny warunek stopu.

22.9 Cwiczenie 8: jawny blok zasięgu

Cel: zadeklaruj zmienna `x` na zewnątrz, a potem utwórz jawny blok `{ ... }`, w którym pojawi się kolejne `x`. Sprawdź przez `print(...)`, że obie wartości są od siebie niezależne.

22.10 Jak pracować z ćwiczeniami

Najlepiej wykonywać je w kolejności. Po każdym z nich odpowiedz sobie na trzy pytania:

1. co już rozumiem lepiej niż przed chwilą,
2. gdzie musiałem użyć `print(...)`,
3. który fragment nadal budzi wątpliwości.

To pomaga nie tylko rozwiązywać zadania, ale też świadomie budować intuicję językową.

Rozdział 23

Jak rozwijac projekt od malego do sredniego serwisu

23.1 Etap 1: jedna strona i jeden chunk

Kazdy projekt warto zaczac od minimalnego pionowego przekroju. Jedna strona, jeden chunk, jeden punkt `moon(...)`. Celem nie jest od razu architektura idealna, tylko pierwszy poprawny wynik HTML.

23.2 Etap 2: wspolne metadata

Gdy pojawia sie druga strona, zwykle od razu warto wydzielic chunk z metadata wspolnymi. To pierwszy realny krok w kierunku reuzywalnosci.

23.3 Etap 3: funkcje pomocnicze

Gdy te same obliczenia zaczynaja powtarzac sie w kilku miejscach, wydziel funkcje. Na tym etapie MoonChunk zaczyna przynosic realny zysk: projekt nie tylko generuje HTML, ale robi to w sposób uporządkowany.

23.4 Etap 4: dane z JSON

Kiedy tresc strony przestaje byc recznie wpisywana, przychodzi pora na `data(...)`. To moment, w ktorym projekt przestaje byc tylko zestawem statycznych napisow, a zaczyna stac sie malym systemem generowania dokumentow na podstawie danych.

23.5 Etap 5: podzial na pliki

Jesli plik glowny zaczyna byc zbyt duzy, rozdziel odpowiedzialnosci:

- metadata,
- strony glowne,
- logika bloga,
- sekcje wspolne,
- dane.

W tym momencie importy staja sie naturalna czescia codziennej pracy.

23.6 Etap 6: wzorce projektowe

Kiedy projekt ma juz kilka stron i kilka rodzajow danych, warto swiadomie wprowadzic powtarzalne wzorce:

- sekcje budowane z tablic obiektow,
- funkcje etykietujace,
- wspolne metadata z lokalnym nadpisaniem,
- jawne punkty wejscia `moon(...)` dla wybranych przebiegow generacji.

23.7 Etap 7: czyszczenie i stabilizacja

Na tym etapie warto przejrzec projekt i zadac kilka pytan kontrolnych:

- czy nazwy sa nadal czytelne,
- czy logika nie rozlala sie do HTML-a,
- czy globali nie jest za duzo,
- czy importy oddaja prawdziwy podzial odpowiedzialnosci,
- czy debugowe `print(...)` nadal sa potrzebne.

23.8 Etap 8: swiadome skalowanie

MoonChunk nie musi rosnac przez doklejanie kolejnych przypadkowych plikow. Dobrze prowadzony projekt rozwija sie warstwowo. Każda nowa strona albo sekcja znajduje swoje miejsce w istniejacej strukturze.

23.9 Zasada, ktora ratuje projekt

Najwazniejsza rada na etapie wzrostu jest prosta: **najpierw nazywaj i porzadkuj, potem dopiero rozbudowuj**. W wielu projektach to wlasnie brak porzadku, a nie brak funkcji, staje sie prawdziwym problemem.

Rozdział A

Szybka sciaga skladni

Ten dodatek nie zastępuje pełnych rozdziałów. Ma służyć jako szybkie przypomnienie najważniejszych konstrukcji.

A.1 Podstawowy szkielet

```
1 chunk "Main" {
2   output: "./dist";
3
4   page "" {
5     content {
6       <h1>Hello</h1>
7     };
8   };
9 };
10
11 moon(Main);
```

A.2 Zmienne i stale

```
1 let a: int = 1;
2 let b: string;
3 b = "hello";
4 const title: string = "MoonChunk";
```

A.3 Warunki

```
1  if (a > 0) {  
2    print("plus");  
3  } else {  
4    print("zero lub minus");  
5  };
```

A.4 Petle

```
1  for (let i: int = 0; i < 10; i++) {  
2    print(i);  
3  };  
4  
5  while (a < 5) {  
6    a = a + 1;  
7  };
```

A.5 Funkcje

```
1  function add(a: int, b: int): int {  
2    return a + b;  
3  }  
4  
5  sum(a: int, b: int): int => a + b;
```

A.6 Importy

```
1  import { Metadata } from "./metadata.mncnk";  
2  import * as Fragments from "./fragments.mncnk";
```

A.7 Include

```
1 chunk "Main" {  
2   @include Metadata;  
3   @include Fragments.Hero;  
4   output: "./dist";  
5 };
```

A.8 Tablice i obiekty

```
1 const arr = [1, 2, 3];  
2 const obj = { title: "Hello", views: 10 };  
3 print(arr[0]);  
4 print(obj.title);
```

A.9 Rzutowanie

```
1 let a: int = "42" as int;  
2 let b: string = 10 as string;  
3 let c: bool = (bool)3;  
4 let d: bool = 3 as unknown as bool;
```

A.10 Globale

```
1 env {  
2   global siteName: string = "MoonChunk";  
3 };
```

Rozdział B

Referencja metadata

Ten rozdział zbiera pola metadata wspierane przez MoonChunk i podpowiada, kiedy ich używać. Nie wszystkie będą potrzebne w każdym projekcie. Warto jednak wiedzieć, że język wspiera szeroki zakres konfiguracji dokumentu.

Pole	Znaczenie i typowe zastosowanie
lang	Język dokumentu, np. "pl" albo "en". Wpływa m.in. na interpretację treści przez przeglądarkę i narzędzia wspomagające.
dir	Kierunek tekstu, zwykle "ltr" dla języków pisanych od lewej do prawej.
htmlClass	Klasy CSS przypisywane do znacznika <html>. Przydatne do globalnego sterowania wyglądem.
charset	Kodowanie znaków, zwykle "utf-8". W praktyce wartość niemal obowiązkowa.
viewport	Ustawienia widoku dla urządzeń mobilnych, np. "width=device-width, initial-scale=1".
title	Tytuł dokumentu pokazywany na karcie przeglądarki i wykorzystywany przez wyszukiwarki.
metaDescription	Krótki opis strony dla SEO i podglądu w wynikach wyszukiwania.
metaKeywords	Lista słów kluczowych. Współcześnie ma mniejsze znaczenie niż dawniej, ale nadal może porządkować projekt.
metaAuthor	Autor strony albo treści.
metaRobots	Wskazówki dla robotów indeksujących, np. "index, follow".
themeColor	Kolor motywu wykorzystywany przez niektóre przeglądarki i urządzenia mobilne.

Pole	Znaczenie i typowe zastosowanie
canonicalUrl	Kanoniczny adres URL strony. Przydatny, gdy jedna treść występuje pod wieloma adresami.
faviconHref	Ścieżka do favikony strony.
appleTouchIconHref	Ikona dla urządzeń Apple.
manifestHref	Ścieżka do manifestu aplikacji webowej.
ogType	Typ Open Graph, np. "website" albo "article".
ogTitle	Tytuł widoczny przy udostępnianiu strony w social media.
ogDescription	Opis strony dla Open Graph.
ogImage	Obraz do podglądu przy udostępnianiu.
ogUrl	Kanoniczny adres strony dla Open Graph.
ogSiteName	Nazwa serwisu.
ogLocale	Lokalizacja Open Graph, np. "pl_PL".
twitterCard	Typ karty Twitter/X, np. "summary" albo "summary_large_image".
twitterSite	Konto serwisu w mediach społecznościowych.
twitterCreator	Konto autora treści.
twitterTitle	Tytuł do karty Twitter/X.
twitterDescription	Opis do karty Twitter/X.
twitterImage	Obraz do karty Twitter/X.
preloadLinks	Własne znaczniki <code><link rel='preload' ...></code> wstawiane do <code><head></code> .
preconnectLinks	Własne znaczniki <code><link rel='preconnect' ...></code> .
styles	Linki do CSS lub inne style, zwykle jako gotowy fragment HTML.
headScripts	Skrypty umieszczane w sekcji <code><head></code> .
headExtra	Dodatkowy surowy HTML dla <code><head></code> .
bodyClass	Klasy CSS dla znacznika <code><body></code> .
pageId	Identyfikator strony trafiający do atrybutu danych w ciele dokumentu.
topBar	Fragment HTML wstawiany nad główną treścią.
header	Fragment HTML nagłówka strony.
footer	Fragment HTML stopki strony.
modals	Miejsce na kontenery modali i innych elementów nakładanych.
scripts	Skrypty umieszczane pod koniec dokumentu, zwykle w <code><body></code> .
bodyEndExtra	Dodatkowy HTML wstawiany na końcu <code><body></code> .

B.1 Praktyczna rada

Nie ma potrzeby uzupełniać wszystkich pól od razu. Zaczynij od kilku najważniejszych: `lang`, `charset`, `viewport`, `title`, `metaDescription`, `styles`. Potem rozbudowuj dokument o kolejne informacje w miarę potrzeb projektu.

Rozdział C

Słownik pojęć

Argument

Wartość przekazywana do funkcji podczas jej wywołania.

Chunk

Podstawowy blok organizacyjny MoonChunk. Może zawierać strony, metadane, funkcje, pętle, warunki i inne instrukcje.

Content

Blok zawierający HTML, który trafi do głównej treści strony.

Deklaracja

Moment utworzenia nowej nazwy, np. przez `let`, `const` albo `global`.

Diagnostyka

Komunikaty o błędach i problemach pojawiających się podczas parsowania lub wykonania.

Global

Wspólna wartość deklarowana w bloku `env`, dostępna w szerszym zakresie niż zwykle zmienne lokalne.

HTML

Język znaczników opisujący strukturę dokumentu strony internetowej.

Import

Mechanizm pobierania chunków z innego pliku `.mncnk`.

Include

Mechanizm dołączania jednego chunku do drugiego przez `@include`.

Interpolacja

Wstawianie dynamicznej wartości do tekstu lub HTML-a, np. przez zapis `{title}`.

JSON

Popularny format zapisu danych, z którego MoonChunk może wczytywać informacje przez `data(...)`.

Metadata

Dodatkowe informacje o dokumencie HTML, zwykle trafiające do sekcji `<head>` albo sterujące ogólną strukturą strony.

Page

Instrukcja opisująca pojedynczą stronę do wygenerowania.

Parametr

Nazwa zdefiniowana w deklaracji funkcji, do której trafia argument przy wywołaniu.

Rekurencja

Sytuacja, w której funkcja wywołuje sama siebie.

Rzutowanie

Świadome przekształcenie lub interpretacja wartości jako innego typu.

Scope / zasięg

Obszar programu, w którym dana nazwa jest widoczna i dostępna.

Static site

Serwis zbudowany z gotowych plików HTML, CSS i JavaScript, bez potrzeby dynamicznego renderowania po stronie serwera przy każdym wejściu użytkownika.

Typ

Informacja o rodzaju wartości, np. liczba, tekst, wartość logiczna, tablica czy obiekt.

Warunek

Wyrażenie logiczne, które decyduje, czy dany blok kodu ma zostać wykonany.

Wyrażenie

Fragment kodu, który po obliczeniu daje wartość.

Zakonczenie

MoonChunk zostal zaprojektowany jako jezyk czytelny, przewidywalny i praktyczny. Jego sila nie wynika z ogromnej liczby funkcji, ale z tego, ze laczy kilka prostych pomyslow: strony, tresc HTML, zmienne, petle, funkcje, importy oraz dane. Razem pozwalaja one zbudowac kompletny, statyczny serwis bez potrzeby siegania po duze, zlozone frameworki.

Najlepsza droga dalszej nauki jest prosta:

1. uruchom male przyklady,
2. przepisuj je samodzielnie,
3. zmieniaj pojedyncze linie i obserwuj efekt,
4. zbuduj swoj pierwszy mini-projekt od zera.

Wlasnie do tego ma sluzyc ten podrecznik.